LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 4

VIEWMODEL AND DEBUGGING

Oleh:

Sebastian Abe Santoso NIM. 2410817210002

PROGRAM STUDI TEKNOLOGI INFORMASI

FAKULTAS TEKNIK

UNIVERSITAS LAMBUNG MANGKURAT

MEI 2026

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN MOBILE
MODUL 4

Laporan Praktikum Pemrograman Mobile Modul 4: ViewModel dan Debugging ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Sebastian Abe Santoso
NIM : 2410817210002

Menyetujui, Asisten Praktikum	Mengetahui, Dosen Penanggung Jawab Praktikum
Muhammad Erza Raihan NIM. 2310817210027	Irham Maulani Abdul Gani, S.Kom., M.Kom. NIP. 199710312025061009

DAFTAR ISI

LEMBAR PENGESAHAN 2
DAFTAR ISI 3
DAFTAR GAMBAR 4
DAFTAR TABEL 5
SOAL 1 6
A. Source Code 6
B. Output Program 59
C. Pembahasan 62
SOAL 2 81
A. Jawaban: 81
TAUTAN GITHUB 82

DAFTAR GAMBAR

Gambar 1. Screenshot Debugging Timber Compose 59
Gambar 2. Screenshot Debugging Timber XML 59
Gambar 3. Screenshot Debugging Step Over Compose 60
Gambar 4. Screenshot Debugging Step Into Compose 60
Gambar 5. Screenshot Debugging Step Out Compose 60
Gambar 6. Screenshot Debugging Step Over XML 61
Gambar 7. Screenshot Debugging Step Into XML 61
Gambar 8. Screenshot Debugging Step Out XML 61

DAFTAR TABEL

Tabel 1. Source Code Main Activity.kt Compose 6
Tabel 2. Source Code Model.kt Compose 7
Tabel 3. Source Code activity_main.xml 8
Tabel 4. Source Code ComicViewModel.kt Compose 16
Tabel 5. Source Code Screen.kt Compose 17
Tabel 6. Source Code ComicViewModelFactory.kt Compose 23
Tabel 7. Source Code ComicRepository.kt Compose 23
Tabel 8. Source Code MainActivity.kt XML 28
Tabel 9. Source Code Comic.kt XML 29
Tabel 10. Source Code MainFragment.kt XML 29
Tabel 11. Source Code DetailsFragment.kt XML 32
Tabel 12. Source Code SettingsFragment.kt XML 35
Tabel 13. Source Code ComicViewModel.kt XML 36
Tabel 14. Source Code CarouselAdapter.kt XML 37
Tabel 15. Source Code ComicListAdapter.kt XML 39
Tabel 16. Source Code ComicViewModelFactory.kt XML 42
Tabel 17. Source Code ComicRepository.kt XML 43
Tabel 18. Source Code activity_main.xml 47
Tabel 19. Source Code fragment_main.xml 48
Tabel 20. Source Code fragment_details.xml 49
Tabel 21. Source Code fragment_settings.xml 52
Tabel 22. Source Code item_carousel.xml 53
Tabel 23. Source Code item_comic_list.xml 56

SOAL 1

Lanjutkan aplikasi Android berbasis XML dan Jetpack Compose yang sudah dibuat pada Modul 3 dengan menambahkan modifikasi sesuai ketentuan berikut:

a. Buatlah sebuah ViewModel untuk menyimpan dan mengelola data dari list item. Data tidak boleh disimpan langsung di dalam Fragment atau Activity.
b. Gunakan ViewModelFactory untuk membuat parameter dengan tipe data String di dalam ViewModel
c. Gunakan StateFlow untuk mengelola event onClick dan data list item dari ViewModel ke Fragment
d. Install dan gunakan library Timber untuk logging event berikut:
    a. Log saat data item masuk ke dalam list
    b. Log saat tombol Detail dan tombol Explicit Intent ditekan
    c. Log data dari list yang dipilih ketika berpindah ke halaman Detail
e. Gunakan tool Debugger di Android Studio untuk melakukan debugging pada aplikasi.

Cari setidaknya satu breakpoint yang relevan dengan aplikasi. Lalu, gunakan fitur Step Into, Step Over, dan Step Out. Setelah itu, jelaskan fungsi Debugger, cara menggunakan Debugger, serta fitur Step Into, Step Over, dan Step Out

A. Source Code

Tabel 1. Source Code Main Activity.kt Compose

No	Main Activity.kt (Compose)
1	package com.example.modul4compose
2	
3	import android.content.res.Configuration
4	import android.os.Bundle
5	import androidx.activity.compose.setContent
6	import androidx.activity.enableEdgeToEdge
7	import androidx.appcompat.app.AppCompatActivity
8	import androidx.compose.ui.platform.LocalConfiguration
9	import androidx.lifecycle.viewmodel.compose.viewModel
10	import androidx.navigation.compose.NavHost
11	import androidx.navigation.compose.composable
12	import
	androidx.navigation.compose.rememberNavController

```
import com.example.modul4compose.ui.theme.Modul3ComposeTheme

class MainActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?)
{
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        setContent {
            Modul3ComposeTheme(darkTheme = true) {
                val navController = rememberNavController()
                val viewModel: ComicViewModel = viewModel(
                    factory = ComicViewModelFactory(
                        appName = "MyComicList",
                        repository = ComicRepository()
                    )
                )

                val isLandscape = LocalConfiguration.current.orientation == Configuration.ORIENTATION_LANDSCAPE

                NavHost(navController, startDestination = "main") {
                    composable("main") {
                        MainScreen(navController, viewModel, isLandscape)
                    }
                    composable ("details"){
                        DetailsScreen (navController, viewModel, isLandscape)
                    }
                    composable("settings"){
                        SettingsScreen(navController)
                    }
                }
            }
        }
    }
}
```

Tabel 2. Source Code Model.kt Compose

No	Model.kt (Compose)

```
1  package com.example.modul4compose
2
3  data class Comic (
4      val title: String,
5      val coverImage: Int,
6      val backgroundImage: Int,
7      val url: String,
8      val description: List<Int>,
9      val genres: List<Int>,
10     val author: String,
11     val id: Int
12 )
```

Tabel 3. Source Code activity_main.xml

No	View.kt (Compose)
1	package com.example.modul4compose
2	
3	import android.content.Intent
4	import androidx.compose.foundation.Image
5	import androidx.compose.foundation.background
6	import androidx.compose.foundation.border
7	import androidx.compose.foundation.clickable
8	import androidx.compose.foundation.gestures.snapping.rememberSnapFlingBehavior
9	import androidx.compose.foundation.layout.Arrangement
10	import androidx.compose.foundation.layout.Box
11	import androidx.compose.foundation.layout.Column
12	import androidx.compose.foundation.layout.Row
13	import androidx.compose.foundation.layout.fillMaxHeight
14	import androidx.compose.foundation.layout.fillMaxSize
15	import androidx.compose.foundation.layout.fillMaxWidth
16	import androidx.compose.foundation.layout.height
17	import androidx.compose.foundation.layout.padding
18	import androidx.compose.foundation.layout.size
19	import androidx.compose.foundation.lazy.LazyRow
20	import androidx.compose.foundation.lazy.items
21	import androidx.compose.foundation.lazy.rememberLazyListState
22	import androidx.compose.foundation.shape.RoundedCornerShape
23	import androidx.compose.material.icons.Icons
24	import androidx.compose.material.icons.filled.*Menu*

```
25 import androidx.compose.material3.Button
26 import androidx.compose.material3.ButtonDefaults
27 import androidx.compose.material3.Card
28 import androidx.compose.material3.Icon
29 import androidx.compose.material3.Text
30 import androidx.compose.runtime.Composable
31 import androidx.compose.ui.Modifier
32 import androidx.compose.ui.draw.clip
33 import androidx.compose.ui.graphics.Brush
34 import androidx.compose.ui.graphics.Color
35 import androidx.compose.ui.layout.ContentScale
36 import androidx.compose.ui.platform.LocalContext
37 import androidx.compose.ui.res.painterResource
38 import androidx.compose.ui.res.stringResource
39 import androidx.compose.ui.text.font.FontWeight
40 import androidx.compose.ui.text.style.TextAlign
41 import androidx.compose.ui.text.style.TextOverflow
42 import androidx.compose.ui.unit.TextUnit
43 import androidx.compose.ui.unit.dp
44 import androidx.compose.ui.unit.em
45 import androidx.compose.ui.unit.sp
46 import androidx.core.net.toUri
47 import androidx.navigation.NavHostController
48
49 @Composable
50 fun RecommendationCarousel(
51     items: List<Comic>,
52     isLandscape: Boolean,
53     navController: NavHostController,
54     viewModel: ComicViewModel){
55     val listState = rememberLazyListState()
56     val flingBehavior =
   rememberSnapFlingBehavior(lazyListState = listState)
57     val descFontSize =  if (isLandscape) 24.sp else
   15.sp
58     val descMaxLine = if (isLandscape) 2 else 6
59
60     val titleFontSize = if (isLandscape) 32.sp else
   20.sp
61     val titleMaxLine = 1
62     val titleLineHeight = 10.sp
63
64     val authorFontSize = if (isLandscape) 24.sp else
   20.sp
65     val authorMaxLine = 1
66
```

```
 val genreFontSize = if (isLandscape) 20.sp else 15.sp
    val genreMaxLine = 1



    val cardHeight = if (isLandscape) 280.dp else 300.dp

    LazyRow(modifier = Modifier
            .fillMaxWidth()
            .fillMaxHeight(),
        state = listState,
        flingBehavior = flingBehavior
    ) {
        items(items){item ->
            Card(modifier = Modifier
                    .fillParentMaxWidth()
                    .height(cardHeight),
                RoundedCornerShape(0.dp)
            ) {
                Box(modifier = Modifier.fillMaxSize()){
                    Image(painter = painterResource(item.backgroundImage),
                        modifier = Modifier.fillMaxSize(),
                        contentScale = ContentScale.Crop,
                        contentDescription = stringResource(R.string.cd_carousel_bg_image))

                    Box(modifier = Modifier
                        .fillMaxSize()
                        .background(
                            brush = Brush.verticalGradient(
                                colors = listOf(
                                Color.Black.copy(alpha = 0.2f),
                                Color.Black.copy(alpha = 0.9f)
                                )
                            )
                        )
                    )

```

```
                    Column(modifier = Modifier.fillMaxSize()) {
                        Text(text = stringResource(R.string.label_recommendation),
                            fontSize = 30.sp,
                            fontWeight = FontWeight.Bold,
                            color = Color.White,
                            modifier = Modifier.padding(vertical = 10.dp, horizontal = 20.dp))

                        ComicCard(item,
                            titleFontSize,
                            titleMaxLine,
                            titleLineHeight,
                            descFontSize,
                            descMaxLine,
                            authorFontSize,
                            authorMaxLine,
                            genreFontSize,
                            genreMaxLine,
                            navController,
                            viewModel,
                        )
                    }
                }
            }
        }
    }
}

@Composable
fun ComicList(
    item: Comic,
    isLandscape: Boolean,
    navController: NavHostController,
    viewModel: ComicViewModel
){
    val descFontSize =  if (isLandscape) 24.sp else 15.sp
    val descMaxLine = if (isLandscape) 2 else 3

    val titleFontSize = if (isLandscape) 32.sp else 20.sp
    val titleMaxLine = 1
```

```
    val titleLineHeight = 10.sp

    val authorFontSize = if (isLandscape) 24.sp else 20.sp
    val authorMaxLine = 1

    val genreFontSize = if (isLandscape) 20.sp else 15.sp
    val genreMaxLine = 1

    val buttonFontSize = if (isLandscape) 30.sp else 18.sp
    val buttonPadding = if (isLandscape) 20.dp else 15.dp
    val buttonIconSize = if (isLandscape) 32.dp else 24.dp
    val cardHeight = if (isLandscape) 280.dp else 300.dp

    Card(modifier = Modifier
        .fillMaxWidth()
        .height(cardHeight)
        .padding(top = 10.dp, bottom = 20.dp)
        .border(2.dp, Color.DarkGray, shape = RoundedCornerShape(10.dp))

    ) {
        val context = LocalContext.current

        Box(modifier = Modifier.fillMaxSize()){
            Image(painter = painterResource(item.backgroundImage),
                modifier = Modifier.fillMaxSize(),
                contentScale = ContentScale.Crop,
                contentDescription = stringResource(R.string.cd_carousel_bg_image))

            Box(modifier = Modifier
                .fillMaxSize()
                .background(
                    brush = Brush.verticalGradient(
                        colors = listOf(
                            Color.Black.copy(alpha = 0.2f),
                            Color.Black.copy(alpha = 0.9f)
                        )
```

```
                    )
                )
            )

            Column(modifier = Modifier.fillMaxSize()) {
                Row(modifier = Modifier.fillMaxSize().weight(1f)
                ){
                    ComicCard(item,
                    titleFontSize,
                    titleMaxLine,
                    titleLineHeight,
                    descFontSize,
                    descMaxLine,
                    authorFontSize,
                    authorMaxLine,
                    genreFontSize,
                    genreMaxLine,
                    navController,
                    viewModel,
                    ) }

                Row(modifier = Modifier.background(Color.Black)
                    .fillMaxWidth().weight(0.5f)
                    .padding(vertical = buttonPadding, horizontal = 15.dp),
                    horizontalArrangement = Arrangement.spacedBy(8.dp)
                ) {
                    Button(onClick = { val intent = Intent(Intent.ACTION_VIEW,
                        item.url.toUri())
                        context.startActivity(intent)
                    }, modifier = Modifier.fillMaxWidth()
                        .fillMaxHeight(0.85f)
                        .weight(1f)
                        .padding(horizontal = 6.dp),
                        shape = RoundedCornerShape(10.dp),
                        colors = ButtonDefaults.buttonColors(containerColor = Color(0xFFFF9800))) {
                        Image(painter = painterResource(R.drawable.mangadex_logo),
```

```
                                contentDescription =
stringResource(R.string.cd_mangadex_logo),
                                contentScale =
ContentScale.Fit,
                                modifier =
Modifier.size(buttonIconSize))
                            Text(text =
stringResource(R.string.btn_mangadex),
                                modifier =
Modifier.padding(start = 10.dp),
                                color = Color.Black,
                                fontSize = buttonFontSize)
                        }

                        Button(onClick = {
navController.navigate("details")
                            viewModel.selectComic(item)
                        }, modifier =
Modifier.fillMaxWidth()
                            .fillMaxHeight(0.85f)
                            .weight(1f)
                            .padding(horizontal = 6.dp),
                            shape =
RoundedCornerShape(10.dp),
                        ) {
                            Icon(Icons.Default.Menu,
                                contentDescription =
stringResource(R.string.cd_detail_icon),
                                modifier =
Modifier.size(buttonIconSize))
                            Text(text =
stringResource(R.string.btn_detail),
                                modifier =
Modifier.padding(start = 10.dp),
                                color = Color.Black,
                                fontSize = buttonFontSize)
                        }
                    }
                }
            }
        }
}

@Composable
fun ComicCard(
    item: Comic,
    titleFontSize: TextUnit,
```

```
    titleMaxLine: Int,
    titleLineHeight: TextUnit,
    descFontSize: TextUnit,
    descMaxLine: Int,
    authorFontSize: TextUnit,
    authorMaxLine: Int,
    genreFontSize: TextUnit,
    genreMaxLine: Int,
    navController: NavHostController,
    viewModel: ComicViewModel,
    ){

    val genresText = item.genres.map {
stringResource(it) }.joinToString(", ")
    val descriptionText = item.description.map {
stringResource(it) }.joinToString("\n\n")

    Row(modifier = Modifier.fillMaxWidth()) {
        Image(painter =
painterResource(item.coverImage),
            modifier = Modifier
                .fillMaxWidth(0.4f)
                .fillMaxHeight()
                .padding(start = 20.dp)
                .clip(RoundedCornerShape(10))
                .clickable{
                    viewModel.selectComic(item)
                    navController.navigate("details")
                },
            contentScale = ContentScale.Crop,

            contentDescription =
stringResource(R.string.cd_cover_image)
        )

        Column(modifier = Modifier
            .fillMaxHeight()
            .fillMaxWidth()
            .padding(10.dp)
            .clickable{
                viewModel.selectComic(item)
                navController.navigate("details")
                }
            ) {
            Text(text = item.title,
                color = Color.White,
                fontSize = titleFontSize,
```

```
293                 fontWeight = FontWeight.Bold,
294                 maxLines = titleMaxLine,
295                 overflow = TextOverflow.Ellipsis,
296                 lineHeight = titleLineHeight
297             )
298             Text(text = descriptionText,
299                 color = Color.White,
300                 fontSize = descFontSize,
301                 maxLines = descMaxLine,
302                 overflow = TextOverflow.Ellipsis,
303                 lineHeight = 1.5.em,
304                 textAlign = TextAlign.Justify
305             )
306
307             Column(Modifier.fillMaxSize(),
                    verticalArrangement =
308 Arrangement.Bottom
309             ) {
310                 Text(text = item.author,
311                     color = Color.White,
312                     fontSize = authorFontSize,
313                     maxLines = authorMaxLine,
314                     overflow = TextOverflow.Ellipsis,
                        modifier =
315 Modifier.padding(vertical = 5.dp)
316                 )
317
                    Row(modifier =
318 Modifier.fillMaxWidth(),
                        horizontalArrangement =
319 Arrangement.SpaceBetween) {
320                     Text(text = genresText,
321                         color = Color.White,
322                         fontSize = genreFontSize,
323                         maxLines = genreMaxLine,
                            overflow =
324 TextOverflow.Ellipsis
325                     )
326                 }
327             }
328         }
329     }
330 }
```

Tabel 4. Source Code ComicViewModel.kt Compose

No	ComicViewModel.kt (Compose)

```
1  package com.example.modul4compose
2
3  import androidx.lifecycle.ViewModel
4  import kotlinx.coroutines.flow.MutableStateFlow
5  import kotlinx.coroutines.flow.asStateFlow
6  import timber.log.Timber
7
8  class ComicViewModel(
9      private val appName: String,
10     private val repository: ComicRepository
11 ): ViewModel() {
       private val _carouselComics =
12 MutableStateFlow<List<Comic>>(emptyList())
       private val _listComics =
13 MutableStateFlow<List<Comic>>(emptyList())
14
       private val _selectedComic =
15 MutableStateFlow<Comic?>(null)
16     val carouselComics = _carouselComics.asStateFlow()
17     val listComics = _listComics.asStateFlow()
18
19     val selectedComic = _selectedComic.asStateFlow()
20
21     fun selectComic(comic: Comic) {
22         Timber.d("Comic dipilih: $comic ")
23         _selectedComic.value = comic
24     }
25
       init {
26         _listComics.value = repository.getListComics()
27         Timber.d("Data item masuk ke dalam list
   ${_listComics.value}")
28
29         _carouselComics.value =
   repository.getCarouselComics()
30         Timber.d("Data item masuk ke dalam list
31 ${_carouselComics.value}")
32     }
33 }
```

Tabel 5. Source Code Screen.kt Compose

No	Screen.kt (Compose)
1	package com.example.modul4compose
2	
3	import androidx.appcompat.app.AppCompatDelegate
4	import androidx.core.os.LocaleListCompat

```
import androidx.compose.foundation.Image
import androidx.compose.foundation.background
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.Column
import
androidx.compose.foundation.layout.fillMaxHeight
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items
import
androidx.compose.foundation.shape.RoundedCornerShape
import androidx.compose.material.icons.Icons
import androidx.compose.material.icons.filled.Menu
import androidx.compose.material3.Button
import
androidx.compose.material3.ExperimentalMaterial3Api
import androidx.compose.material3.Icon
import androidx.compose.material3.IconButton
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.Scaffold
import androidx.compose.material3.Text
import androidx.compose.material3.TopAppBar
import androidx.compose.runtime.Composable
import androidx.compose.runtime.getValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Brush
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.layout.ContentScale
import androidx.compose.ui.res.painterResource
import androidx.compose.ui.res.stringResource
import androidx.compose.ui.text.style.TextAlign
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import
androidx.lifecycle.compose.collectAsStateWithLifecycle
import androidx.navigation.NavHostController

@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun MainScreen(
    navController: NavHostController,
    viewModel: ComicViewModel,
    isLandscape: Boolean
```

```
47 ){
48     val listComics by
49 viewModel.listComics.collectAsStateWithLifecycle()
    val carouselComics by
50 viewModel.carouselComics.collectAsStateWithLifecycle()

51
52     Scaffold(
53         modifier = Modifier.fillMaxSize(),
54         topBar = {
55             TopAppBar(
56                 title = {
57
Text(stringResource(R.string.app_title),
58                         style =
MaterialTheme.typography.titleLarge
59                     )},
60                 actions = { IconButton(onClick =
{navController.navigate("settings") }) {
61                         Icon(Icons.Default.Menu,
62                             contentDescription =
stringResource(R.string.cd_settings_menu))
63                     }
64                 }
65             )
66         }
67     ) {innerPadding ->
68
69         LazyColumn(modifier = Modifier
70             .fillMaxSize()
71             .padding(innerPadding)
72         ) {
73             item {
74                 RecommendationCarousel(carouselComics,
75                     isLandscape,
76                     navController,
77                     viewModel)
78             }
79
80             item {
81                 Text(text =
stringResource(R.string.label_top_comic),
82                 style =
MaterialTheme.typography.titleMedium,
83                 fontSize = 25.sp,
84                 modifier = Modifier.padding(top =
30.dp, bottom = 5.dp, start = 20.dp)
```

```
                )
            }



            items(listComics) {item ->
                ComicList(item, isLandscape,
navController, viewModel)
            }
        }
    }
}

@Composable
fun DetailsScreen(
    navController: NavHostController,
    viewModel: ComicViewModel,
    isLandscape: Boolean){
    val selectedComic by
viewModel.selectedComic.collectAsStateWithLifecycle()
    val descFontSize =  if (isLandscape) 24.sp else
20.sp
    val descMaxLine = if (isLandscape) 3 else 15

    val titleFontSize = if (isLandscape) 32.sp else
40.sp
    val titleMaxLine = if (isLandscape) 1 else 10
    val titleLineHeight = 50.sp

    val authorFontSize = if (isLandscape) 24.sp else
20.sp
    val authorMaxLine = if (isLandscape) 1 else 3

    val genreFontSize = 20.sp
    val genreMaxLine = if (isLandscape) 1 else 3

    val buttonFontSize = if (isLandscape) 30.sp else
20.sp
    val cardHeight = if (isLandscape) 0.75f else 0.89f
    Scaffold(
        modifier = Modifier.fillMaxSize()
    ) {innerPadding ->
        selectedComic?.let { comic ->
            Box(modifier = Modifier.fillMaxSize()){
                Image(painter =
painterResource(comic.backgroundImage),
                    modifier = Modifier.fillMaxSize(),
```

```
                    contentScale = ContentScale.Crop,
                    contentDescription =
stringResource(R.string.cd_bg_image))

                Box(modifier = Modifier
                    .fillMaxSize()
                    .background(
                        brush =
Brush.verticalGradient(
                            colors = listOf(
                                Color.Black.copy(alpha
= 0.2f),
                                Color.Black.copy(alpha
= 0.9f)
                            )
                        )
                    )
                )
                Column (modifier = Modifier
                    .padding(innerPadding)
                    .fillMaxSize())
                {
                    Column(modifier = Modifier
                        .fillMaxWidth()
                        .fillMaxHeight(cardHeight))
{ComicCard(comic, titleFontSize, titleMaxLine,
titleLineHeight, descFontSize, descMaxLine,
authorFontSize, authorMaxLine, genreFontSize,
genreMaxLine, navController, viewModel)
                    }
                    Button(onClick = {
navController.popBackStack()
                    }, modifier = Modifier
                        .fillMaxSize()
                        .padding(20.dp), shape =
RoundedCornerShape(10.dp)) {
                        Text(text =
stringResource(R.string.btn_return), modifier =
Modifier.padding(end = 10.dp), fontSize =
buttonFontSize, textAlign = TextAlign.Center)
                    }
                }
            }
        }
    }
}
```

```
@Composable
fun SettingsScreen(navController: NavHostController) {
    val currentLang = AppCompatDelegate.getApplicationLocales()[0]?.language ?: “en”

    val newLang = if (currentLang == “en”) “id” else “en”

    Scaffold(
        modifier = Modifier.fillMaxSize()
    ) { innerPadding ->

        Column(modifier = Modifier
                .padding(innerPadding)
                .fillMaxSize(),
            verticalArrangement = Arrangement.SpaceEvenly,
            horizontalAlignment = Alignment.CenterHorizontally
        ) {

            Text(text = stringResource(R.string.label_change_language),
                style = MaterialTheme.typography.titleLarge,
                modifier = Modifier.padding(bottom = 20.dp)
            )

            Button(onClick = {
                AppCompatDelegate.setApplicationLocales(
                LocaleListCompat.forLanguageTags(newLang))
                },
                modifier = Modifier
                    .fillMaxHeight(0.5f)
                    .fillMaxWidth(),
                shape = RoundedCornerShape(10.dp)
            ) {
                Text(text = stringResource(R.string.btn_switch_language),
                    textAlign = TextAlign.Center
                )
            }

```

```
193             Button(onClick = {
194 navController.popBackStack() },
195                 modifier = Modifier
196                     .fillMaxHeight(0.5f)
197
198                     .fillMaxWidth(),
                    shape = RoundedCornerShape(10.dp)
199             ) {
200                 Text(text =
201 stringResource(R.string.btn_return),
202                     textAlign = TextAlign.Center
203                 )
204             }
        }
    }
}
```

Tabel 6. Source Code ComicViewModelFactory.kt Compose

No	ComicViewModelFactory.kt (Compose)
1	package com.example.modul4compose
2	
3	import androidx.lifecycle.ViewModel
4	import androidx.lifecycle.ViewModelProvider
5	
6	class ComicViewModelFactory (
7	private val appName: String,
8	private val repository: ComicRepository
9	): ViewModelProvider.Factory {
10	@Suppress("UNCHECKED_CAST")
11	override fun <T : ViewModel> create(modelClass: Class<T>): T {
12	if (modelClass.isAssignableFrom(ComicViewModel::class.java)){
13	return ComicViewModel(appName, repository) as T
14	}
15	throw IllegalArgumentException("Unknown ViewModel")
16	}
17	}

Tabel 7. Source Code ComicRepository.kt Compose

No	ComicRepository.kt (Compose)
1	package com.example.modul4compose

```
2
3  class ComicRepository {
4      fun getCarouselComics(): List<Comic> {
5          return listOf(
6              Comic(
7                  “Please Bully Me Miss Villainess”,
8                  R.drawable.cv_cr_1,
9                  R.drawable.bg_cr_1,
10                 “https://mangadex.org/title/8b34f37a-
   0181-4f0b-8ce3-01217e9a602c/qing-qifu-wo-ba-eyi-
   xiaojie”,
11 listOf(R.string.please_bully_me_miss_villainess),
12                 listOf(
13                     R.string.genre_romance,
14                     R.string.genre_comedy,
15                     R.string.genre_transmigration
16                 ),
17                 “Chise, Ciweimao (刺猬猫), 断s, Fairy
18 Club (魔仙社)”,
                   1
19             ),
20             Comic(
21                 “Reverend Insanity”,
22                 R.drawable.cv_cr_2,
23                 R.drawable.bg_cr_2,
24                 “https://mangadex.org/title/e1809b04-
   f7dd-4327-9669-ca83884a1403/reverend-insanity-remake”,
25                 listOf(R.string.reverend_insanity),
26                 listOf(R.string.genre_action,
   R.string.genre_adventure, R.string.genre_wuxia),
27                 “Gu Zhen Ren, Rococo (Kodoku Studio),
   LenHC, Z (Kodoku Studio)”,
28                 2
29             ),
30             Comic(
31                 “SSS-Class Suicide Hunter”,
32                 R.drawable.cv_cr_3,
33                 R.drawable.bg_cr_3,
34                 “https://mangadex.org/title/4a973243-
   952e-44d7-a50f-883b4b7c9cc2/sss-geup-jugeoya-saneun-
   hunter”,
35             listOf(R.string.sss_class_suicide_hunter),
36                 listOf(R.string.genre_action,
   R.string.genre_adventure, R.string.genre_time_travel),
37
38                 “Neida (네이다), Shin Noah (신노아),
   Bill K”,
```

```
                3
            ),
            Comic(
                "I Got a New Skill Every Time I was Exiled",
                R.drawable.cv_cr_4,
                R.drawable.bg_cr_4,
                "https://mangadex.org/title/5f64d517-c4ea-40c0-a463-fcc069d39d3b/tsuihou-sareru-tabi-ni-skill-o-te-ni-ireta-ore-ga-100-no-isekai-de-2-shuume-musou",
listOf(R.string.i_got_a_new_skill_every_time_i_was_exiled),
                listOf(
                    R.string.genre_reincarnation,
                    R.string.genre_time_travel,
                    R.string.genre_action,
                    R.string.genre_romance
                ),
                "Hinoura Takumi, Nimori Shimatsukasa",
                4
            ),
            Comic(
                "The Greatest Estate Developer",
                R.drawable.cv_cr_5,
                R.drawable.bg_cr_5,
                "https://mangadex.org/title/d7f56ace-cd30-48b9-8b64-afeca0077fca/yeokdaegeum-yeongji-seolgyesa",
listOf(R.string.the_greatest_estate_developer),
                listOf(R.string.genre_action, R.string.genre_comedy, R.string.genre_reincarnation),
                "Lee Hyunmin (이현민), BK_Moon (문백경), Kim Hyeon-Soo (김현수)",
                5
            ),
            Comic(
                "Lord of the Mysteries",
                R.drawable.cv_cr_6,
                R.drawable.bg_cr_6,
                "https://mangadex.org/title/df3f7834-a65b-4d2c-9388-045c9ec03a35/lord-of-mysteries",
               listOf(R.string.lord_of_the_mysteries),
                listOf(
                    R.string.genre_transmigration,
                    R.string.genre_historical,
```

```
                    R.string.genre_action
                ),
                “Cuttlefish That Loves Diving (爱潜水的乌贼), sevenballoon”,
                6
            ),
            Comic(
                “The Summer You Were There”,
                R.drawable.cv_cr_7,
                R.drawable.bg_cr_7,
                “https://mangadex.org/title/6ecc62e4-25ad-4102-b0d8-580a8023d2fb/kimi-to-tsuzuru-utakata”,
            listOf(R.string.the_summer_you_were_there),
                listOf(
                    R.string.genre_psychological,
                    R.string.genre_tragedy,
                    R.string.genre_romance
                ),
                “Yuama”,
                7
            ),
        )
    }
    fun getListComics(): List<Comic> {
        return listOf(
            Comic(
                “The Moon on a Rainy Night”,
                R.drawable.cv_li_1,
                R.drawable.bg_li_1,
                “https://mangadex.org/title/c8a0426d-539b-4f6a-9537-fbccf4962529/amayo-no-tsuki”,
            listOf(R.string.the_moon_on_a_rainy_night),
                listOf(R.string.genre_romance, R.string.genre_drama, R.string.genre_school_life),
                “Kuzushiro”,
                8),
            Comic(
                “Movies Are Real”,
                R.drawable.cv_li_2,
                R.drawable.bg_li_2,
                “https://mangadex.org/title/ff2f35ff-2641-4a1b-8477-df933efd4eba/movies-are-real”,
                listOf(R.string.movies_are_real),
                listOf(R.string.genre_action, R.string.genre_psychological, R.string.genre_mystery),

```

```
                “Jaemihalgi (재미할기), Toon Plus (주)툰플러스, AinJam (아인잠)”,
                9),
            Comic(
                “Legend of The Northern Blade”,
                R.drawable.cv_li_3,
                R.drawable.bg_li_3,
                “https://mangadex.org/title/9ed16bc9-f570-4e71-8dda-aebc098b683b/the-legend-of-the-northern-blade”,
        listOf(R.string.legend_of_the_northern_blade),
                listOf(R.string.genre_action, R.string.genre_martial_arts, R.string.genre_adventure),
                “Ugak (우각), Haemin (해민)”,
                10),
            Comic(
                “Return of the Mount Hua Sect”,
                R.drawable.cv_li_4,
                R.drawable.bg_li_4,
                “https://mangadex.org/title/f0f62b75-5989-4f32-9b59-ab56abe35fc1/return-of-the-blossoming-blade”,
        listOf(R.string.return_of_the_mount_hua_sect),
                listOf(R.string.genre_reincarnation, R.string.genre_time_travel, R.string.genre_martial_arts),
                “Biga (비가), LICO”,
                11),
            Comic(
                “The One Within the Villainess”,
                R.drawable.cv_li_5,
                R.drawable.bg_li_5,
                “https://mangadex.org/title/6df29d13-2dab-4ca6-a0b5-74070abf3e1d/akuyaku-reijou-no-naka-no-hito”,
listOf(R.string.the_one_within_the_villainess),
                listOf(R.string.genre_reincarnation, R.string.genre_action, R.string.genre_psychological),
                “Makiburo, Shiraume Nazuna”,
                12),
            Comic(
                “The King’s Avatar”,
                R.drawable.cv_li_6,
                R.drawable.bg_li_6,
```

```
                “https://mangadex.org/title/1930d635-
148 b170-417f-b8a8-f84b881bcc7d/quanzhi-gaoshou”,
149                listOf(R.string.the_kings_avatar),
                listOf(R.string.genre_action,
150 R.string.genre_comedy, R.string.genre_fantasy),
                “Blue Butterfly, Yuewen Animation &
151 Comics”,
152                13),
153            Comic(
                “I Made Friends with the Second
154 Prettiest Girl in My Class”,
155                R.drawable.cv_li_7,
156                R.drawable.bg_li_7,
                “https://mangadex.org/title/8ef11280-
    30bc-434b-bf61-c61e092905ac/class-de-2-banme-ni-
    kawaii-onnanoko-to-tomodachi-ni-natta”,
    listOf(R.string.i_made_friends_with_the_second_prettie
157 st_girl_in_my_class),
                listOf(R.string.genre_romance,
158 R.string.genre_comedy, R.string.genre_school_life),
159                “Takata, Ono Rin”,
160                14),
161        )
162    }
    }
```

Tabel 8. Source Code MainActivity.kt XML

No	MainActivity.kt (XML)
1	package com.example.modul4xml
2	
3	import android.os.Bundle
4	import androidx.activity.enableEdgeToEdge
5	import androidx.appcompat.app.AppCompatActivity
6	import androidx.core.view.ViewCompat
7	import androidx.core.view.WindowInsetsCompat
8	import com.example.modul4xml.databinding.ActivityMainBinding
9	
10	class MainActivity : AppCompatActivity() {
11	private lateinit var binding: ActivityMainBinding
12	
13	override fun onCreate(savedInstanceState: Bundle?) {
14	super.onCreate(savedInstanceState)
15	enableEdgeToEdge()
16	
17	

```
        binding =
18 ActivityMainBinding.inflate(layoutInflater)
19
20         setContentView(binding.root)
21
   ViewCompat.setOnApplyWindowInsetsListener(binding.navHos
   tFragment) { v, insets ->
22             val systemBars =
   insets.getInsets(WindowInsetsCompat.Type.systemBars())
23             v.setPadding(systemBars.left,
   systemBars.top, systemBars.right, systemBars.bottom)
24             insets
25         }
26     }
27 }
```

Tabel 9. Source Code Comic.kt XML

No	Comic.kt (XML)
1	package com.example.modul4xml.model
2	
3	data class Comic(
4	val title: String,
5	val coverImage: Int,
6	val backgroundImage: Int,
7	val url: String,
8	val description: List<Int>,
9	val genres: List<Int>,
10	val author: String,
11	val id: Int
12	)

Tabel 10. Source Code MainFragment.kt XML

No	MainFragment.kt (XML)
1	package com.example.modul4xml.fragment
2	
3	import android.content.res.Configuration
4	import android.os.Bundle
5	import android.view.LayoutInflater
6	import android.view.View
7	import android.view.ViewGroup
8	import androidx.fragment.app.Fragment
9	import androidx.fragment.app.activityViewModels
10	import androidx.lifecycle.Lifecycle
11	import androidx.lifecycle.lifecycleScope
12	

```
13 import androidx.lifecycle.repeatOnLifecycle
14 import androidx.navigation.fragment.findNavController
15 import androidx.recyclerview.widget.LinearLayoutManager
16 import androidx.recyclerview.widget.PagerSnapHelper
17 import com.example.modul4xml.R
18 import com.example.modul4xml.adapter.CarouselAdapter
19 import com.example.modul4xml.adapter.ComicListAdapter
   import
20 com.example.modul4xml.databinding.FragmentMainBinding
   import
21 com.example.modul4xml.factory.ComicViewModelFactory
22 import com.example.modul4xml.repository.ComicRepository
23 import com.example.modul4xml.viewmodel.ComicViewModel
24 import kotlinx.coroutines.launch
25
26 class MainFragment : Fragment() {
27     private var _binding: FragmentMainBinding? = null
28     private val binding get() = _binding!!
29     private val viewModel: ComicViewModel by
   activityViewModels {
30 ComicViewModelFactory(
31 appName = “Modul 3 XML”,
32 repository = ComicRepository()
33 )
34 }
35
36     override fun onCreateView(
37         inflater: LayoutInflater,
38         container: ViewGroup?,
39         savedInstanceState: Bundle?
40     ): View {
         _binding = FragmentMainBinding.inflate(inflater,
41 container, false)
42         return binding.root
43     }
44
    override fun onViewCreated(view: View,
45 savedInstanceState: Bundle?) {
46         super.onViewCreated(view, savedInstanceState)
47
         val isLandscape =
   resources.configuration.orientation ==
48 Configuration.ORIENTATION_LANDSCAPE
49         val navController = findNavController()
50
         binding.toolbar.setOnMenuItemClickListener {
   menuItem ->
```

```
            if (menuItem.itemId == R.id.action_settings) {
51
52 navController.navigate(R.id.action_main_to_settings)
                true
53              } else false
54          }
55
56          val carouselAdapter =
   CarouselAdapter(isLandscape, viewModel, navController)
57          binding.carouselRecyclerView.apply {
58              layoutManager =
59                  LinearLayoutManager(requireContext(),
   LinearLayoutManager.HORIZONTAL, false)
60              adapter = carouselAdapter
61              PagerSnapHelper().attachToRecyclerView(this)
62          }
63
64          if (isLandscape) {
65              val lp =
   binding.carouselRecyclerView.layoutParams
66              lp.height =
   resources.getDimensionPixelSize(R.dimen.carousel_height_
   landscape)
67              binding.carouselRecyclerView.layoutParams =
   lp
68          }
69
70          val listAdapter = ComicListAdapter(isLandscape,
   viewModel, navController)
71          binding.listRecyclerView.apply {
72              layoutManager =
   LinearLayoutManager(requireContext())
73              adapter = listAdapter
74              isNestedScrollingEnabled = false
75          }
76
77      viewLifecycleOwner.lifecycleScope.launch {
78 viewLifecycleOwner.repeatOnLifecycle(Lifecycle.State.STA
   RTED) {
79 viewModel.carouselComics.collect { comics ->
80
81 carouselAdapter.submitList(comics)
82 }
83 }
84          }
85
86 viewLifecycleOwner.lifecycleScope.launch {
```

```
87 viewLifecycleOwner.repeatOnLifecycle(Lifecycle.State.STA
   RTED) {
88 viewModel.listComics.collect { comics ->
89
90 listAdapter.submitList(comics)
91 }
   }
           }
92     }
93 override fun onDestroyView() {
94 super.onDestroyView()
95 _binding = null
96 }
97 }
98
99
```

Tabel 11. Source Code DetailsFragment.kt XML

No	DetailsFragment.kt (XML)
1	package com.example.modul4xml.fragment
2	
3	import android.content.res.Configuration
4	import android.os.Bundle
5	import android.view.LayoutInflater
6	import android.view.View
7	import android.view.ViewGroup
8	import androidx.fragment.app.Fragment
9	import androidx.fragment.app.activityViewModels
10	import androidx.lifecycle.Lifecycle
11	import androidx.lifecycle.lifecycleScope
12	import androidx.lifecycle.repeatOnLifecycle
13	import androidx.navigation.fragment.findNavController
14	import com.example.modul4xml.databinding.FragmentDetailsBinding
15	import com.example.modul4xml.factory.ComicViewModelFactory
16	import com.example.modul4xml.repository.ComicRepository
17	import com.example.modul4xml.viewmodel.ComicViewModel
18	import kotlinx.coroutines.launch
19	
20	class DetailsFragment : Fragment() {
21	private var _binding: FragmentDetailsBinding? = null
	private val binding get() = _binding!!

```
    private val viewModel: ComicViewModel by
22 activityViewModels {
23         ComicViewModelFactory(
               appName = “Modul 3 XML”,
24             repository = ComicRepository()
25         )
26     }
27
28     override fun onCreateView(
29         inflater: LayoutInflater,
30         container: ViewGroup?,
31         savedInstanceState: Bundle?
32
33     ): View {
34         _binding =
35 FragmentDetailsBinding.inflate(inflater, container,
   false)

36         return binding.root
37     }
38
39     override fun onViewCreated(view: View,
40 savedInstanceState: Bundle?) {
           super.onViewCreated(view, savedInstanceState)
41
42         val isLandscape =
43 resources.configuration.orientation ==
   Configuration.ORIENTATION_LANDSCAPE
           val navController = findNavController()
44
45         val cardWeight = if (isLandscape) 75f else 89f
46         val buttonWeight = if (isLandscape) 25f else
47 11f

48         val cardLp =
49 binding.comicCardArea.layoutParams as
   android.widget.LinearLayout.LayoutParams
           cardLp.weight = cardWeight
50         binding.comicCardArea.layoutParams = cardLp
51
52         val btnLp = binding.btnReturn.layoutParams as
53 android.widget.LinearLayout.LayoutParams
           btnLp.weight = buttonWeight
54         binding.btnReturn.layoutParams = btnLp
55
56         val titleFontSize = if (isLandscape) 32f else
57 40f
```

```
        val titleMaxLines = if (isLandscape) 1 else 10
        val descFontSize = if (isLandscape) 24f else 20f
        val descMaxLines = if (isLandscape) 3 else 15
        val authorFontSize = if (isLandscape) 24f else 20f
        val authorMaxLines = if (isLandscape) 1 else 3
        val genreFontSize = 20f
        val genreMaxLines = if (isLandscape) 1 else 3
        val buttonFontSize = if (isLandscape) 30f else 20f
        viewLifecycleOwner.lifecycleScope.launch {
viewLifecycleOwner.repeatOnLifecycle(Lifecycle.State.STARTED) {
                viewModel.selectedComic.collect {
comic ->
                    comic ?: return@collect

                    val ctx = requireActivity()

binding.backgroundImage.setImageResource(comic.backgroundImage)
binding.coverImage.setImageResource(comic.coverImage)

                    binding.titleText.apply {
                        text = comic.title
                        textSize = titleFontSize
                        maxLines = titleMaxLines
                    }
                    binding.descriptionText.apply {
                        text =
comic.description.joinToString("\n\n") {
ctx.getString(it) }
                        textSize = descFontSize
                        maxLines = descMaxLines
                    }
                    binding.authorText.apply {
                        text = comic.author
                        textSize = authorFontSize
                        maxLines = authorMaxLines
                    }
                    binding.genreText.apply {
                        text =
comic.genres.joinToString(", ") { ctx.getString(it) }
                        textSize = genreFontSize
                        maxLines = genreMaxLines
                    }
```

```
                    }
            }
        }

        binding.btnReturn.apply {
            textSize = buttonFontSize
            setOnClickListener {
navController.popBackStack() }
        }
    }

    override fun onDestroyView() {
        super.onDestroyView()
        _binding = null
    }
}
```

Tabel 12. Source Code SettingsFragment.kt XML

No	SettingsFragment.kt (XML)
1	package com.example.modul4xml.fragment
2	
3	import android.os.Bundle
4	import android.view.LayoutInflater
5	import android.view.View
6	import android.view.ViewGroup
7	import androidx.appcompat.app.AppCompatDelegate
8	import androidx.core.os.LocaleListCompat
9	import androidx.fragment.app.Fragment
10	import androidx.navigation.fragment.findNavController
11	import com.example.modul4xml.databinding.FragmentSettingsBinding
12	
13	class SettingsFragment : Fragment() {
14	private var _binding: FragmentSettingsBinding? =
15	null
16	private val binding get() = _binding!!
17	override fun onCreateView(
18	inflater: LayoutInflater,
19	container: ViewGroup?, savedInstanceState: Bundle?

```
20      ): View {
21          _binding =
22  FragmentSettingsBinding.inflate(inflater, container,
23  false)
24          return binding.root
25      }
26
        override fun onViewCreated(view: View,
    savedInstanceState: Bundle?) {
27          super.onViewCreated(view, savedInstanceState)
28
29          val navController = findNavController()
30
            binding.btnSwitchLanguage.setOnClickListener {
31              val currentLocales =
32  AppCompatDelegate.getApplicationLocales()
33              val currentLang = if
34  (!currentLocales.isEmpty) {
35                  currentLocales[0]?.language
36              } else {
                    java.util.Locale.getDefault().language
37              }

38              val newLang = if (currentLang == "id" ||
39  currentLang == "in") "en" else "id"
40
41  AppCompatDelegate.setApplicationLocales(LocaleListCompat
42  .forLanguageTags(newLang))
43          }

44          binding.btnReturn.setOnClickListener {
45              navController.popBackStack()
            }
46      }
47
48      override fun onDestroyView() {
49          super.onDestroyView()
50          _binding = null
51      }
52  }
```

Tabel 13. Source Code ComicViewModel.kt XML

No	**ComicViewModel.kt (XML)**
1	package com.example.modul4xml.viewmodel
2	
3	import androidx.lifecycle.ViewModel

```
4  import com.example.modul4xml.model.Comic
5  import
   com.example.modul4xml.repository.ComicRepository
6  import kotlinx.coroutines.flow.MutableStateFlow
7  import kotlinx.coroutines.flow.StateFlow
8  import kotlinx.coroutines.flow.asStateFlow
9  import timber.log.Timber
10 import kotlin.collections.emptyList
11
12 class ComicViewModel(
13     private val appName: String,
14     private val repo: ComicRepository
15 ) : ViewModel() {
       private val _carouselComics =
16 MutableStateFlow<List<Comic>>(emptyList())
       private val _listComics =
17 MutableStateFlow<List<Comic>>(emptyList())
       private val _selectedComic =
18 MutableStateFlow<Comic?>(null)
19
       val carouselComics: StateFlow<List<Comic>> =
20 _carouselComics.asStateFlow()
       val listComics: StateFlow<List<Comic>> =
21 _listComics.asStateFlow()
       val selectedComic: StateFlow<Comic?> =
22 _selectedComic.asStateFlow()
23
24     fun selectComic(comic: Comic) {
25         Timber.d("Comic dipilih: $comic ")
26         _selectedComic.value = comic
27     }
28
29     init {
30         _listComics.value = repo.getListComics()
31         Timber.d("Data item masuk ke dalam list
   ${_listComics.value}")
32
33         _carouselComics.value =
   repo.getCarouselComics()
34         Timber.d("Data item masuk ke dalam list
   ${_carouselComics.value}")
35     }
36 }
```

Tabel 14. Source Code CarouselAdapter.kt XML

No	CarouselAdapter.kt (XML)

```
1  package com.example.modul4xml.adapter
2
3  import android.view.LayoutInflater
4  import android.view.ViewGroup
5  import androidx.navigation.NavController
6  import androidx.recyclerview.widget.DiffUtil
7  import androidx.recyclerview.widget.ListAdapter
8  import androidx.recyclerview.widget.RecyclerView
9  import com.example.modul4xml.R
10 import
   com.example.modul4xml.databinding.ItemCarouselBinding
11 import com.example.modul4xml.model.Comic
12 import com.example.modul4xml.viewmodel.ComicViewModel
13
14 class CarouselAdapter(
15     private val isLandscape: Boolean,
16     private val viewModel: ComicViewModel,
17     private val navController: NavController
18 ) : ListAdapter<Comic,
   CarouselAdapter.ViewHolder>(DiffCallback()) {
19
20     class ViewHolder(val binding: ItemCarouselBinding) :
21         RecyclerView.ViewHolder(binding.root)
22
23     override fun onCreateViewHolder(parent: ViewGroup,
   viewType: Int): ViewHolder {
24         val binding = ItemCarouselBinding.inflate(
25             LayoutInflater.from(parent.context), parent,
   false
26         )
27         return ViewHolder(binding)
28     }
29
30     override fun onBindViewHolder(holder: ViewHolder,
   position: Int) {
31         val comic = getItem(position)
32         val ctx = holder.itemView.context
33
34         with(holder.binding) {

35 backgroundImage.setImageResource(comic.backgroundImage)
36            coverImage.setImageResource(comic.coverImage)
37
38             titleText.apply {
39                 text = comic.title
40                 textSize = if (isLandscape) 32f else 20f
41                 maxLines = 1
```

```
42              }
43              descriptionText.apply {
44                  text =
    comic.description.joinToString("\n\n") {
    ctx.getString(it) }
45                  textSize = if (isLandscape) 24f else 15f
46                  maxLines = if (isLandscape) 2 else 5
47              }
48              authorText.apply {
49                  text = comic.author
50                  textSize = if (isLandscape) 24f else 20f
51                  maxLines = 1
52              }
53              genreText.apply {
54                  text = comic.genres.joinToString(", ") {
    ctx.getString(it) }
55                  textSize = if (isLandscape) 20f else 15f
56                  maxLines = 1
57              }
58
59              val navigate = {
60                  viewModel.selectComic(comic)
61
    navController.navigate(R.id.action_main_to_details)
62              }
63              coverImage.setOnClickListener { navigate() }
64              infoColumn.setOnClickListener { navigate() }
65          }
66      }
67
68      class DiffCallback : DiffUtil.ItemCallback<Comic>()
    {
69          override fun areItemsTheSame(oldItem: Comic,
    newItem: Comic) = oldItem.id == newItem.id
70          override fun areContentsTheSame(oldItem: Comic,
    newItem: Comic) = oldItem == newItem
71      }
72  }
```

Tabel 15. Source Code ComicListAdapter.kt XML

No	ComicListAdapter.kt (XML)
1	package com.example.modul4xml.adapter
2	
3	import android.content.Intent
4	import android.view.LayoutInflater
5	import android.view.ViewGroup

```
import androidx.core.net.toUri
import androidx.navigation.NavController
import androidx.recyclerview.widget.DiffUtil
import androidx.recyclerview.widget.ListAdapter
import androidx.recyclerview.widget.RecyclerView
import com.example.modul4xml.R
import com.example.modul4xml.databinding.ItemComicListBinding
import com.example.modul4xml.model.Comic
import com.example.modul4xml.viewmodel.ComicViewModel

class ComicListAdapter(
    private val isLandscape: Boolean,
    private val viewModel: ComicViewModel,
    private val navController: NavController
) : ListAdapter<Comic, ComicListAdapter.ViewHolder>(DiffCallback()) {

    class ViewHolder(val binding: ItemComicListBinding) :
        RecyclerView.ViewHolder(binding.root)

    override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): ViewHolder {
        val binding = ItemComicListBinding.inflate(
            LayoutInflater.from(parent.context), parent, false
        )

        val cardHeightPx = parent.context.resources.getDimensionPixelSize(
            if (isLandscape) R.dimen.list_card_height_landscape
            else R.dimen.list_card_height_portrait
        )
        binding.root.layoutParams = (binding.root.layoutParams as RecyclerView.LayoutParams).also {
            it.height = cardHeightPx
        }
        return ViewHolder(binding)
    }

    override fun onBindViewHolder(holder: ViewHolder, position: Int) {
        val comic = getItem(position)
        val ctx = holder.itemView.context
```

```
43
44          with(holder.binding) {
45
backgroundImage.setImageResource(comic.backgroundImage)
46              coverImage.setImageResource(comic.coverImage)
47
48              titleText.apply {
49                  text = comic.title
50                  textSize = if (isLandscape) 32f else 20f
51                  maxLines = 1
52              }
53              descriptionText.apply {
54                  text =
comic.description.joinToString("\n\n") {
ctx.getString(it) }
55                  textSize = if (isLandscape) 24f else 15f
56                  maxLines = if (isLandscape) 2 else 3
57              }
58              authorText.apply {
59                  text = comic.author
60                  textSize = if (isLandscape) 24f else 20f
61                  maxLines = 1
62              }
63              genreText.apply {
64                  text = comic.genres.joinToString(", ") {
ctx.getString(it) }
65                  textSize = if (isLandscape) 20f else 15f
66                  maxLines = 1
67              }
68
69              val buttonFontSize = if (isLandscape) 30f
else 20f
70
71              btnMangadex.textSize = buttonFontSize
72              btnDetail.textSize = buttonFontSize
73
74              val navigate = {
75                  viewModel.selectComic(comic)
76                  val currentDest =
navController.currentDestination?.id
77
78                  if (currentDest == R.id.mainFragment) {

79 navController.navigate(R.id.action_main_to_details)
80                  }
81              }
82
```

```
83              coverImage.setOnClickListener { navigate() }
84              infoColumn.setOnClickListener { navigate() }
85
86              btnMangadex.setOnClickListener {
                    val intent = Intent(Intent.ACTION_VIEW,
87  comic.url.toUri())
88                  ctx.startActivity(intent)
89              }
90
91              btnDetail.setOnClickListener { navigate() }
92          }
93      }
94
95      class DiffCallback : DiffUtil.ItemCallback<Comic>()
96  {
            override fun areItemsTheSame(oldItem: Comic,
97  newItem: Comic) = oldItem.id == newItem.id
            override fun areContentsTheSame(oldItem: Comic,
98  newItem: Comic) = oldItem == newItem
99      }
    }
```

Tabel 16. Source Code ComicViewModelFactory.kt XML

No	ComicViewModelFactory.kt (XML)
1	package com.example.modul4xml.factory
2	
3	import androidx.lifecycle.ViewModel
4	import androidx.lifecycle.ViewModelProvider
5	import com.example.modul4xml.repository.ComicRepository
6	import com.example.modul4xml.viewmodel.ComicViewModel
7	
8	class ComicViewModelFactory(
9	private val appName: String,
10	private val repository: ComicRepository
11	): ViewModelProvider.Factory {
12	@Suppress("UNCHECKED_CAST")
13	override fun <T : ViewModel> create(modelClass: Class<T>): T {
14	if (modelClass.isAssignableFrom(ComicViewModel::class.java)){
15	return ComicViewModel(appName, repository) as T
16	}
17	throw IllegalArgumentException("Unknown ViewModel")

```
        }
}
```

Tabel 17. Source Code ComicRepository.kt XML

No	ComicRepository.kt (XML)
1	package com.example.modul4compose
2	
3	class ComicRepository {
4	fun getCarouselComics(): List<Comic> {
5	return listOf(
6	Comic(
7	“Please Bully Me Miss Villainess”,
8	R.drawable.cv_cr_1,
9	R.drawable.bg_cr_1,
10	“https://mangadex.org/title/8b34f37a-0181-4f0b-8ce3-01217e9a602c/qing-qifu-wo-ba-eyi-xiaojie”,
11	listOf(R.string.please_bully_me_miss_villainess),
12	listOf(
13	R.string.genre_romance,
14	R.string.genre_comedy,
15	R.string.genre_transmigration
16	),
17	“Chise, Ciweimao (刺猬猫), 断s, Fairy Club (魔仙社)”,
18	1
19	),
20	Comic(
21	“Reverend Insanity”,
22	R.drawable.cv_cr_2,
23	R.drawable.bg_cr_2,
24	“https://mangadex.org/title/e1809b04-f7dd-4327-9669-ca83884a1403/reverend-insanity-remake”,
25	listOf(R.string.reverend_insanity),
26	listOf(R.string.genre_action, R.string.genre_adventure, R.string.genre_wuxia),
27	“Gu Zhen Ren, Rococo (Kodoku Studio), LenHC, Z (Kodoku Studio)”,
28	2
29	),
30	Comic(
31	“SSS-Class Suicide Hunter”,
32	R.drawable.cv_cr_3,
33	R.drawable.bg_cr_3,
34	

```
                "https://mangadex.org/title/4a973243-952e-44d7-a50f-883b4b7c9cc2/sss-geup-jugeoya-saneun-hunter",
            listOf(R.string.sss_class_suicide_hunter),
                listOf(R.string.genre_action, R.string.genre_adventure, R.string.genre_time_travel),
                "Neida (네이다), Shin Noah (신노아), Bill K",
                3
            ),
            Comic(
                "I Got a New Skill Every Time I was Exiled",
                R.drawable.cv_cr_4,
                R.drawable.bg_cr_4,
                "https://mangadex.org/title/5f64d517-c4ea-40c0-a463-fcc069d39d3b/tsuihou-sareru-tabi-ni-skill-o-te-ni-ireta-ore-ga-100-no-isekai-de-2-shuume-musou",
                listOf(R.string.i_got_a_new_skill_every_time_i_was_exiled),
                listOf(
                    R.string.genre_reincarnation,
                    R.string.genre_time_travel,
                    R.string.genre_action,
                    R.string.genre_romance
                ),
                "Hinoura Takumi, Nimori Shimatsukasa",
                4
            ),
            Comic(
                "The Greatest Estate Developer",
                R.drawable.cv_cr_5,
                R.drawable.bg_cr_5,
                "https://mangadex.org/title/d7f56ace-cd30-48b9-8b64-afeca0077fca/yeokdaegeum-yeongji-seolgyesa",
                listOf(R.string.the_greatest_estate_developer),
                listOf(R.string.genre_action, R.string.genre_comedy, R.string.genre_reincarnation),
                "Lee Hyunmin (이현민), BK_Moon (문백경), Kim Hyeon-Soo (김현수)",
                5
            ),
            Comic(
                "Lord of the Mysteries",
```

```
                R.drawable.cv_cr_6,
                R.drawable.bg_cr_6,
                "https://mangadex.org/title/df3f7834-
a65b-4d2c-9388-045c9ec03a35/lord-of-mysteries",
               listOf(R.string.lord_of_the_mysteries),
                listOf(
                    R.string.genre_transmigration,
                    R.string.genre_historical,
                    R.string.genre_action
                ),
                "Cuttlefish That Loves Diving (爱潜水的
乌贼), sevenballoon",
                6
            ),
            Comic(
                "The Summer You Were There",
                R.drawable.cv_cr_7,
                R.drawable.bg_cr_7,
                "https://mangadex.org/title/6ecc62e4-
25ad-4102-b0d8-580a8023d2fb/kimi-to-tsuzuru-utakata",
           listOf(R.string.the_summer_you_were_there),
                listOf(
                    R.string.genre_psychological,
                    R.string.genre_tragedy,
                    R.string.genre_romance
                ),
                "Yuama",
                7
            ),
        )
    }
    fun getListComics(): List<Comic> {
        return listOf(
            Comic(
                "The Moon on a Rainy Night",
                R.drawable.cv_li_1,
                R.drawable.bg_li_1,
                "https://mangadex.org/title/c8a0426d-
539b-4f6a-9537-fbccf4962529/amayo-no-tsuki",
           listOf(R.string.the_moon_on_a_rainy_night),
                listOf(R.string.genre_romance,
R.string.genre_drama, R.string.genre_school_life),
                "Kuzushiro",
                8),
            Comic(
                "Movies Are Real",
```

```
                R.drawable.cv_li_2,
                R.drawable.bg_li_2,
                "https://mangadex.org/title/ff2f35ff-2641-4a1b-8477-df933efd4eba/movies-are-real",
                listOf(R.string.movies_are_real),
                listOf(R.string.genre_action, R.string.genre_psychological, R.string.genre_mystery),
                "Jaemihalgi (재미할기), Toon Plus (주)툰플러스, AinJam (아인잠)",
                9),
            Comic(
                "Legend of The Northern Blade",
                R.drawable.cv_li_3,
                R.drawable.bg_li_3,
                "https://mangadex.org/title/9ed16bc9-f570-4e71-8dda-aebc098b683b/the-legend-of-the-northern-blade",
        listOf(R.string.legend_of_the_northern_blade),
                listOf(R.string.genre_action, R.string.genre_martial_arts, R.string.genre_adventure),
                "Ugak (우각), Haemin (해민)",
                10),
            Comic(
                "Return of the Mount Hua Sect",
                R.drawable.cv_li_4,
                R.drawable.bg_li_4,
                "https://mangadex.org/title/f0f62b75-5989-4f32-9b59-ab56abe35fc1/return-of-the-blossoming-blade",
        listOf(R.string.return_of_the_mount_hua_sect),
                listOf(R.string.genre_reincarnation, R.string.genre_time_travel, R.string.genre_martial_arts),
                "Biga (비가), LICO",
                11),
            Comic(
                "The One Within the Villainess",
                R.drawable.cv_li_5,
                R.drawable.bg_li_5,
                "https://mangadex.org/title/6df29d13-2dab-4ca6-a0b5-74070abf3e1d/akuyaku-reijou-no-naka-no-hito",
listOf(R.string.the_one_within_the_villainess),
```

```
                listOf(R.string.genre_reincarnation,
R.string.genre_action, R.string.genre_psychological),
                “Makiburo, Shiraume Nazuna”,
                12),
            Comic(
                “The King’s Avatar”,
                R.drawable.cv_li_6,
                R.drawable.bg_li_6,
                “https://mangadex.org/title/1930d635-
b170-417f-b8a8-f84b881bcc7d/quanzhi-gaoshou”,
                listOf(R.string.the_kings_avatar),
                listOf(R.string.genre_action,
R.string.genre_comedy, R.string.genre_fantasy),
                “Blue Butterfly, Yuewen Animation &
Comics”,
                13),
            Comic(
                “I Made Friends with the Second
Prettiest Girl in My Class”,
                R.drawable.cv_li_7,
                R.drawable.bg_li_7,
                “https://mangadex.org/title/8ef11280-
30bc-434b-bf61-c61e092905ac/class-de-2-banme-ni-
kawaii-onnanoko-to-tomodachi-ni-natta”,
listOf(R.string.i_made_friends_with_the_second_prettie
st_girl_in_my_class),
                listOf(R.string.genre_romance,
R.string.genre_comedy, R.string.genre_school_life),
                “Takata, Ono Rin”,
                14),
        )
    }
}
```

Tabel 18. Source Code activity_main.xml

No	activity_main.xml

```
<?xml version=“1.0” encoding=“utf-8”?>

<androidx.fragment.app.FragmentContainerView
xmlns:android=“http://schemas.android.com/apk/res/androi
d”
    xmlns:app=“http://schemas.android.com/apk/res-auto”
    android:id=“@+id/nav_host_fragment”
```

```
 7  android:name="androidx.navigation.fragment.NavHostFragme
    nt"
 8      android:layout_width="match_parent"
 9      android:layout_height="match_parent"
10      app:navGraph="@navigation/nav_graph"
11      app:defaultNavHost="true" />
```

Tabel 19. Source Code fragment_main.xml

No	**fragment_main.xml**

```
 1  <?xml version="1.0" encoding="utf-8"?>
 2  <androidx.coordinatorlayout.widget.CoordinatorLayout
 3  xmlns:android="http://schemas.android.com/apk/res/androi
    d"
 4      xmlns:app="http://schemas.android.com/apk/res-auto"
 5      android:layout_width="match_parent"
 6      android:layout_height="match_parent">
 7
 8      <com.google.android.material.appbar.AppBarLayout
 9          android:id="@+id/appBarLayout"
10          android:layout_width="match_parent"
11          android:layout_height="wrap_content">
12
13       <com.google.android.material.appbar.MaterialToolbar
14              android:id="@+id/toolbar"
15              android:layout_width="match_parent"
16              android:layout_height="?attr/actionBarSize"
17              app:title="@string/app_title"
18  app:titleTextAppearance="?attr/textAppearanceTitleLarge"
19              app:menu="@menu/main_menu" />
20
21      </com.google.android.material.appbar.AppBarLayout>
22
23      <androidx.core.widget.NestedScrollView
24          android:layout_width="match_parent"
25          android:layout_height="match_parent"
26  app:layout_behavior="@string/appbar_scrolling_view_behav
    ior"
27          android:overScrollMode="never">
28
29          <LinearLayout
30              android:layout_width="match_parent"
31              android:layout_height="wrap_content"
32              android:orientation="vertical">
33
34              <androidx.recyclerview.widget.RecyclerView
35                  android:id="@+id/carouselRecyclerView"
```

```
36                  android:layout_width="match_parent"
37                  android:layout_height="300dp"
38                  android:orientation="horizontal"
39                  android:overScrollMode="never" />
40
41              <TextView
42                  android:id="@+id/sectionTitle"
43                  android:layout_width="match_parent"
44                  android:layout_height="wrap_content"
45                  android:text="@string/label_top_comic"
46  android:textAppearance="?attr/textAppearanceTitleMedium"
47                  android:textSize="25sp"
48                  android:paddingTop="30dp"
49                  android:paddingBottom="5dp"
50                  android:paddingStart="20dp"
51                  android:paddingEnd="20dp" />
52
53              <androidx.recyclerview.widget.RecyclerView
54                  android:id="@+id/listRecyclerView"
55                  android:layout_width="match_parent"
56                  android:layout_height="wrap_content"
57                  android:nestedScrollingEnabled="false"
58                  android:overScrollMode="never" />
59
60          </LinearLayout>
61      </androidx.core.widget.NestedScrollView>
62
63  </androidx.coordinatorlayout.widget.CoordinatorLayout>
```

Tabel 20. Source Code fragment_details.xml

No	**fragment_details.xml**
1	<?xml version="1.0" encoding="utf-8"?>
2	<FrameLayout
3	xmlns:android="http://schemas.android.com/apk/res/andr oid"
4	xmlns:app="http://schemas.android.com/apk/res-auto"
5	android:layout_width="match_parent"
6	android:layout_height="match_parent"
7	android:fitsSystemWindows="true">
8	
9	<ImageView
10	android:id="@+id/backgroundImage"
11	android:layout_width="match_parent"
12	android:layout_height="match_parent"
13	android:scaleType="centerCrop"

```
14     android:contentDescription="@string/cd_bg_image" />
15
16      <View
17          android:layout_width="match_parent"
18          android:layout_height="match_parent"
19 android:background="@drawable/gradient_card_overlay"/>
20
21      <LinearLayout
22          android:layout_width="match_parent"
23          android:layout_height="match_parent"
24          android:orientation="vertical"
25          android:weightSum="100">
26
27          <LinearLayout
28              android:id="@+id/comicCardArea"
29              android:layout_width="match_parent"
30              android:layout_height="0dp"
31              android:layout_weight="89"
32              android:orientation="horizontal"
33              android:weightSum="10">
34
35 <com.google.android.material.imageview.ShapeableImageV
   iew
36                 android:id="@+id/coverImage"
37                 android:layout_width="0dp"
38                 android:layout_height="match_parent"
39                 android:layout_weight="4"
40                 android:layout_marginStart="20dp"
41                 android:scaleType="centerCrop"
42 android:contentDescription="@string/cd_cover_image"
43 app:shapeAppearanceOverlay="@style/RoundedCoverImage"
   />
44
45             <LinearLayout
46                 android:id="@+id/infoColumn"
47                 android:layout_width="0dp"
48                 android:layout_height="match_parent"
49                 android:layout_weight="6"
50                 android:orientation="vertical"
51                 android:padding="10dp">
52
53                 <TextView
54                     android:id="@+id/titleText"
55                    android:layout_width="match_parent"
56                   android:layout_height="wrap_content"
57               android:textColor="@android:color/white"
58                     android:textStyle="bold"
```

```
59                      android:ellipsize="end" />
60
61                  <TextView
62                      android:id="@+id/descriptionText"
63                     android:layout_width="match_parent"
64                      android:layout_height="0dp"
65                      android:layout_weight="1"
66                 android:textColor="@android:color/white"
67                      android:ellipsize="end"
68                    android:lineSpacingMultiplier="1.5"
69                      android:gravity="top" />
70
71                  <LinearLayout
72                     android:layout_width="match_parent"
73                    android:layout_height="wrap_content"
74                      android:orientation="vertical">
75
76                      <TextView
77                          android:id="@+id/authorText"
78                    android:layout_width="match_parent"
79                    android:layout_height="wrap_content"
80                 android:textColor="@android:color/white"
81                          android:ellipsize="end"
82                          android:paddingTop="5dp"
83                          android:paddingBottom="5dp" />
84
85                      <TextView
86                          android:id="@+id/genreText"
87                    android:layout_width="match_parent"
88                    android:layout_height="wrap_content"
89                 android:textColor="@android:color/white"
90                          android:ellipsize="end" />
91
92                  </LinearLayout>
93              </LinearLayout>
94          </LinearLayout>
95
96      <com.google.android.material.button.MaterialButton
97              android:id="@+id/btnReturn"
98              android:layout_width="match_parent"
99              android:layout_height="0dp"
100             android:layout_weight="11"
101             android:layout_margin="20dp"
102             android:text="@string/btn_return"
103             app:cornerRadius="10dp" />
104
105     </LinearLayout>
```

```
106
107 </FrameLayout>
```

Tabel 21. Source Code fragment_settings.xml

No	fragment_settings.xml

```
1  <?xml version="1.0" encoding="utf-8"?>
2  <LinearLayout
3  xmlns:android="http://schemas.android.com/apk/res/androi
   d"
4      xmlns:app="http://schemas.android.com/apk/res-auto"
5      android:layout_width="match_parent"
6      android:layout_height="match_parent"
7      android:orientation="vertical"
8      android:weightSum="5">
9
10     <Space
11         android:layout_width="match_parent"
12         android:layout_height="0dp"
13         android:layout_weight="1" />
14
15     <TextView
16         android:layout_width="match_parent"
17         android:layout_height="wrap_content"
18         android:text="@string/label_change_language"
19  android:textAppearance="?attr/textAppearanceTitleLarge"
20         android:gravity="center"
21         android:paddingBottom="20dp" />
22
23     <Space
24         android:layout_width="match_parent"
25         android:layout_height="0dp"
26         android:layout_weight="1" />
27
28     <com.google.android.material.button.MaterialButton
29         android:id="@+id/btnSwitchLanguage"
30         android:layout_width="match_parent"
31         android:layout_height="0dp"
32         android:layout_weight="1"
33         android:layout_marginStart="16dp"
34         android:layout_marginEnd="16dp"
35         android:text="@string/btn_switch_language"
36         android:gravity="center"
37         app:cornerRadius="10dp" />
38
```

```
39      <Space
40          android:layout_width="match_parent"
41          android:layout_height="0dp"
42          android:layout_weight="0.5" />
43
44      <com.google.android.material.button.MaterialButton
45          android:id="@+id/btnReturn"
46          android:layout_width="match_parent"
47          android:layout_height="0dp"
48          android:layout_weight="1"
49          android:layout_marginStart="16dp"
50          android:layout_marginEnd="16dp"
51          android:text="@string/btn_return"
52          android:gravity="center"
53          app:cornerRadius="10dp" />
54
55      <Space
56          android:layout_width="match_parent"
57          android:layout_height="0dp"
58          android:layout_weight="0.5" />
59
60  </LinearLayout>
```

Tabel 22. Source Code item_carousel.xml

No	item_carousel.xml
1	<?xml version="1.0" encoding="utf-8"?>
2	<FrameLayout
3	xmlns:android="http://schemas.android.com/apk/res/andr oid"
4	android:layout_width="match_parent"
5	android:layout_height="match_parent">
6	
7	<ImageView
8	android:id="@+id/backgroundImage"
9	android:layout_width="match_parent"
10	android:layout_height="match_parent"
11	android:scaleType="centerCrop"
12	android:contentDescription="@string/cd_carousel_bg_ima ge" />
13	
14	<View
15	android:layout_width="match_parent"
16	android:layout_height="match_parent"
17	android:background="@drawable/gradient_card_overlay" />

```
18      <LinearLayout
19          android:layout_width="match_parent"
20          android:layout_height="match_parent"
21          android:orientation="vertical">
22
23          <TextView
24              android:layout_width="match_parent"
25              android:layout_height="wrap_content"
26             android:text="@string/label_recommendation"
27              android:textSize="30sp"
28              android:textStyle="bold"
29              android:textColor="@android:color/white"
30              android:paddingTop="10dp"
31              android:paddingBottom="10dp"
32              android:paddingStart="20dp"
33              android:paddingEnd="20dp" />
34
35          <LinearLayout
36              android:layout_width="match_parent"
37              android:layout_height="match_parent"
38              android:orientation="horizontal"
39              android:weightSum="10">
40
41  <com.google.android.material.imageview.ShapeableImageV
42  iew
                    android:id="@+id/coverImage"
43                  android:layout_width="0dp"
44                  android:layout_height="match_parent"
45                  android:layout_weight="4"
46                  android:layout_marginStart="20dp"
47                  android:scaleType="centerCrop"
48     android:contentDescription="@string/cd_cover_image"
49  app:shapeAppearanceOverlay="@style/RoundedCoverImage"
50  xmlns:app="http://schemas.android.com/apk/res-auto"/>
51
52              <LinearLayout
53                  android:id="@+id/infoColumn"
54                  android:layout_width="0dp"
55                  android:layout_height="match_parent"
56                  android:layout_weight="6"
57                  android:orientation="vertical"
58                  android:padding="10dp">
59
60                  <TextView
61                      android:id="@+id/titleText"
62                     android:layout_width="match_parent"
63                    android:layout_height="wrap_content"
```

```
64                 android:textColor="@android:color/white"
65                       android:textStyle="bold"
66                       android:maxLines="1"
67                       android:ellipsize="end" />
68
69                   <TextView
70                       android:id="@+id/descriptionText"
71                     android:layout_width="match_parent"
72                       android:layout_height="0dp"
73

74 android:layout_weight="1"
75 android:textColor="@android:color/white"
76                       android:ellipsize="end"
77                      android:lineSpacingMultiplier="1.5"
78                       android:gravity="top" />
79
80                   <LinearLayout
81                      android:layout_width="match_parent"
82                     android:layout_height="wrap_content"
83                       android:orientation="vertical">
84
85                       <TextView
86                           android:id="@+id/authorText"
87                      android:layout_width="match_parent"
88                     android:layout_height="wrap_content"
89                 android:textColor="@android:color/white"
90                           android:maxLines="1"
91                           android:ellipsize="end"
92                           android:paddingTop="5dp"
93                           android:paddingBottom="5dp" />
94
95                       <TextView
96                           android:id="@+id/genreText"
97                      android:layout_width="match_parent"
98                     android:layout_height="wrap_content"
99                 android:textColor="@android:color/white"
100                          android:maxLines="1"
101                          android:ellipsize="end" />
102                  </LinearLayout>
103
104              </LinearLayout>
105          </LinearLayout>
106      </LinearLayout>
107
108 </FrameLayout>
```

Tabel 23. Source Code item_comic_list.xml

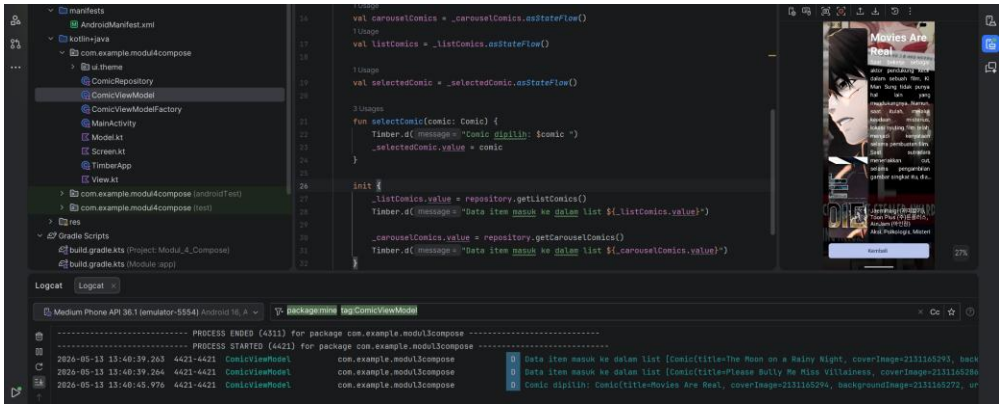
No	item_comic_list.xml
1	<?xml version="1.0" encoding="utf-8"?>
2	<com.google.android.material.card.MaterialCardView
3	xmlns:android="http://schemas.android.com/apk/res/andr oid"
4	xmlns:app="http://schemas.android.com/apk/res- auto"
5	android:layout_width="match_parent"
6	android:layout_height="200dp"
7	android:layout_marginTop="10dp"
8	android:layout_marginBottom="20dp"
9	app:cardCornerRadius="12dp"
10	app:cardElevation="4dp">
11	
12	<FrameLayout
13	android:layout_width="match_parent"
14	android:layout_height="match_parent">
15	
16	<ImageView
17	android:id="@+id/backgroundImage"
18	android:layout_width="match_parent"
19	android:layout_height="match_parent"
20	android:scaleType="centerCrop"
21	android:contentDescription="@string/cd_carousel_bg_ima ge" />
22	
23	<View
24	android:layout_width="match_parent"
25	android:layout_height="match_parent"
26	android:background="@drawable/gradient_card_overlay" />
27	
28	<LinearLayout
29	android:layout_width="match_parent"
30	android:layout_height="match_parent"
31	android:orientation="vertical"
32	android:weightSum="14">
33	
34	<LinearLayout
35	android:layout_width="match_parent"
36	android:layout_height="0dp"
37	android:layout_weight="10"
38	android:orientation="horizontal"
39	android:weightSum="10">
40	
41	

```
<com.google.android.material.imageview.ShapeableImageV
iew
                    android:id="@+id/coverImage"
                    android:layout_width="0dp"
                  android:layout_height="match_parent"
                    android:layout_weight="4"
                    android:layout_marginStart="20dp"
                    android:scaleType="centerCrop"
   android:contentDescription="@string/cd_cover_image"
app:shapeAppearanceOverlay="@style/RoundedCoverImage"
/>

                <LinearLayout
                    android:id="@+id/infoColumn"
                    android:layout_width="0dp"
                  android:layout_height="match_parent"
                    android:layout_weight="6"
                    android:orientation="vertical"
                    android:padding="10dp">

                    <TextView
                        android:id="@+id/titleText"
                   android:layout_width="match_parent"
                  android:layout_height="wrap_content"
              android:textColor="@android:color/white"
                        android:textStyle="bold"
                        android:maxLines="1"
                        android:ellipsize="end" />

                    <TextView
                     android:id="@+id/descriptionText"
                   android:layout_width="match_parent"
                        android:layout_height="0dp"
                        android:layout_weight="1"
              android:textColor="@android:color/white"
                        android:ellipsize="end"
                   android:lineSpacingMultiplier="1.5"
                        android:gravity="top" />

                    <LinearLayout
                   android:layout_width="match_parent"
                  android:layout_height="wrap_content"
                       android:orientation="vertical">

                        <TextView
                          android:id="@+id/authorText"
                   android:layout_width="match_parent"
```

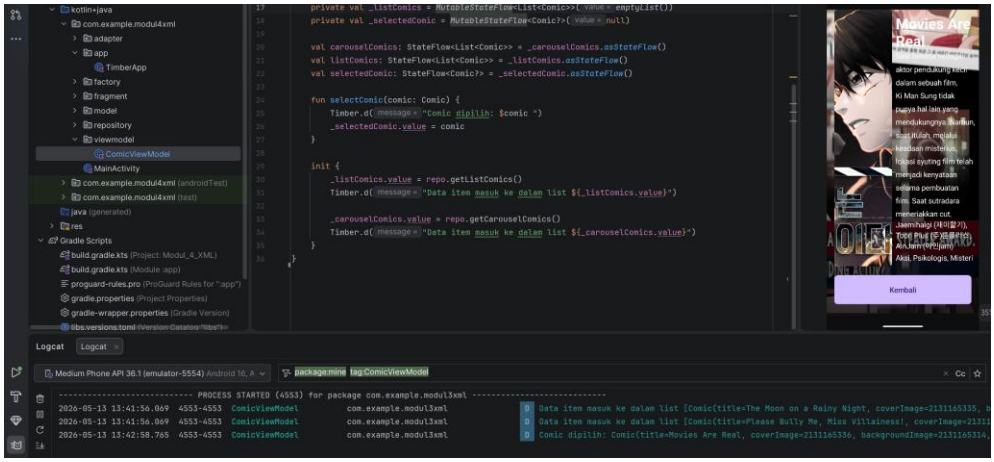
```
                    android:layout_height="wrap_content"
                android:textColor="@android:color/white"
                               android:maxLines="1"
                               android:ellipsize="end"
                               android:paddingTop="5dp"
                           android:paddingBottom="5dp" />

                           <TextView
                              android:id="@+id/genreText"
                     android:layout_width="match_parent"
                    android:layout_height="wrap_content"
                android:textColor="@android:color/white"
                               android:maxLines="1"
                               android:ellipsize="end" />

                      </LinearLayout>
                  </LinearLayout>
              </LinearLayout>

              <LinearLayout
                  android:layout_width="match_parent"
                  android:layout_height="0dp"
                  android:layout_weight="4"
                  android:orientation="horizontal"
               android:background="@android:color/black"
                  android:paddingTop="8dp"
                  android:paddingStart="20dp"
                  android:paddingEnd="20dp"
                  android:paddingBottom="8dp">


      <com.google.android.material.button.MaterialButton
                      android:id="@+id/btnMangadex"
                      android:layout_width="0dp"
                    android:layout_height="match_parent"
                      android:layout_weight="1"
                      android:layout_marginEnd="10dp"
                     android:text="@string/btn_mangadex"
                android:textColor="@android:color/black"
                      app:backgroundTint="#FFFF9800"
                      app:cornerRadius="10dp"
                      app:icon="@drawable/mangadex_logo"
                      app:iconGravity="textStart"
                      app:iconTint="@null" />

      <com.google.android.material.button.MaterialButton
                      android:id="@+id/btnDetail"
```

```
                        android:layout_width="0dp"
                    android:layout_height="match_parent"
                        android:layout_weight="1"
                        android:layout_marginStart="10dp"
                        android:text="@string/btn_detail"
                        android:padding="8dp"
                android:textColor="@android:color/black"
                        app:backgroundTint="#b9c5ed"
                        app:cornerRadius="10dp"
                        app:icon="@drawable/ic_menu_24"
                        app:iconGravity="textStart" />

                </LinearLayout>
            </LinearLayout>
        </FrameLayout>

</com.google.android.material.card.MaterialCardView>
```

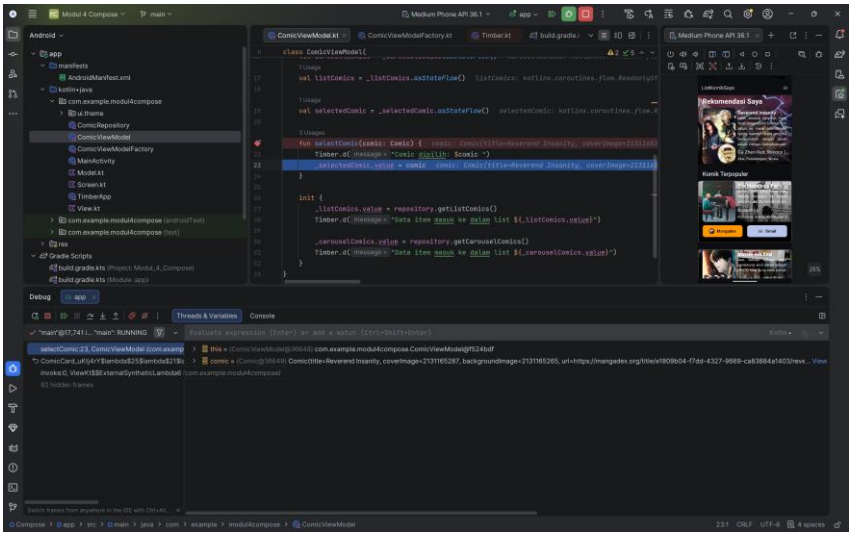
B. Output Program



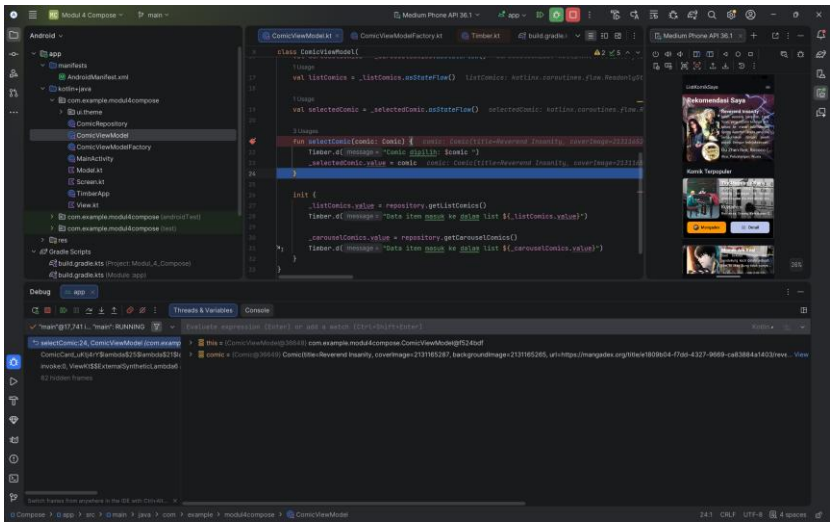
Gambar 1. Screenshot Debugging Timber Compose



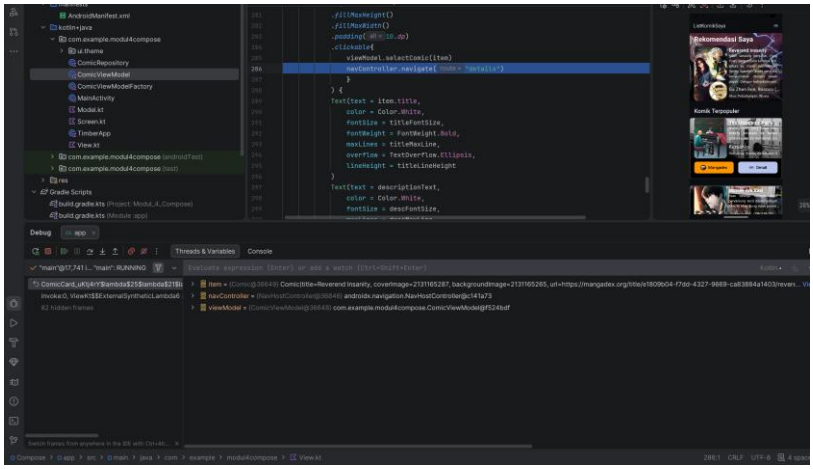
Gambar 2. Screenshot Debugging Timber XML



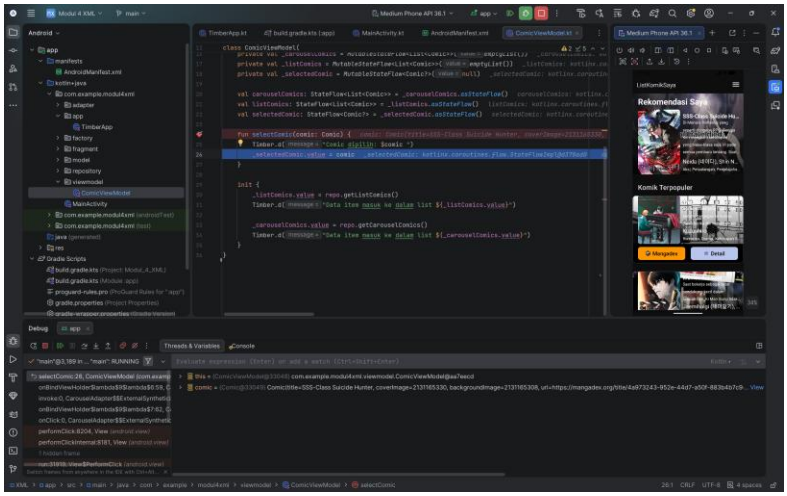
Gambar 3. Screenshot Debugging Step Over Compose



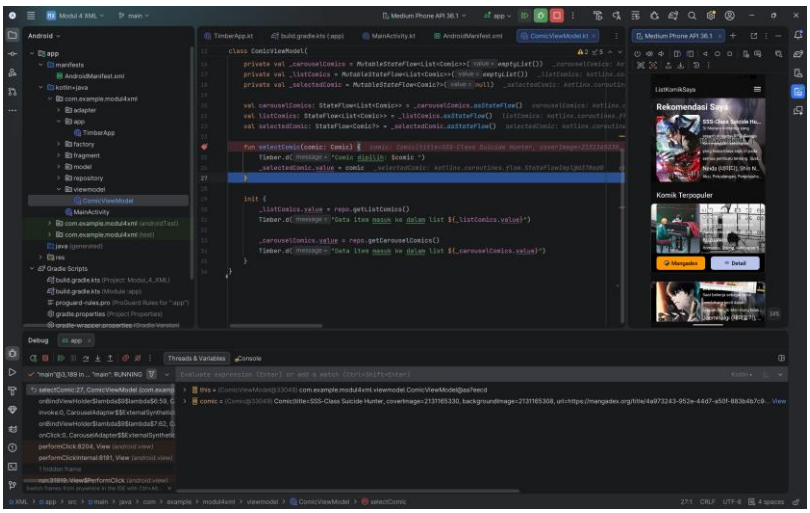
Gambar 4. Screenshot Debugging Step Into Compose



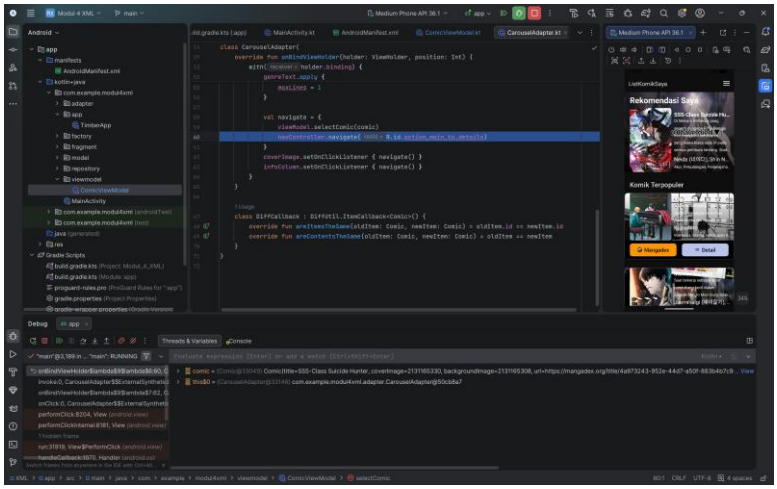
Gambar 5. Screenshot Debugging Step Out Compose



Gambar 6. Screenshot Debugging Step Over XML



Gambar 7. Screenshot Debugging Step Into XML



Gambar 8. Screenshot Debugging Step Out XML

C. Pembahasan

1. Penjelasan mengenai debugger:

- Debugger adalah alat di Android Studio yang digunakan untuk program secara sementara pada titik tertentu, lalu memeriksa alur eksekusi dan nilai variabel saat program sedang berjalan.
- Kegunaan utama debugger adalah untuk mencari dan memahami bug.
- Cara menggunakan:

  1. Tentukan breakpoint/bagian kode yang ingin diperiksa
  2. Klik area kiri di nomor baris untuk menambahkan breakpoint
  3. Jalankan aplikasi menggunakan debug, di sebelah run.
  4. Jalankan aksi pada aplikasi sampai kode yang memiliki breakpoint terpanggil.
  5. Ketika breakpoint terkena, program akan berhenti sementara.
  6. Saat berhenti, terlihat nilai variabel, call stack, dan menjalankan kode baris per baris menggunakan fitur Step Into, Step Over, dan Step Out.

  Sebagai contoh, breakpoint yang saya gunakan adalah selctComic() karena saya rasa itu bagian penting, saat user meng-klik komik untuk melihat detailnya.

  Step Into berguna untuk masuk ke dalam fungsi yang sedang dipanggil, digunakan ketika developer ingin melihat proses internal dari sebuah fungsi secara lebih detail.

  Step Over digunakan untuk menjalankan baris kode saat ini tanpa masuk ke dalam fungsi yang dipanggil, berguna jika developer ingin melewati satu baris kode dan melihat hasil akhirnya.

  Step Out adalah fitur debugger untuk keluar dari fungsi yang sedang diperiksa dan kembali ke fungsi pemanggil sebelumnya, berguna ketika developer sudah selesai memeriksa isi sebuah fungsi dan ingin melanjutkan proses debugging di alur sebelumnya.

2. Penjelasan Kode:

Model.kt (COMPOSE)

Pada baris [1] terdapat deklarasi package ‘com.example.modul4compose’, yang menandakan lokasi file ini dalam package project.

Pada baris [3] sampai [12] terdapat ‘data classComic’, yang berfungsi sebagai Model dalam arsitektur MVVM, yaitu representasi data dari sebuah komik. Sesuai peran di MVVM, class ini hanya berisi struktur data.

- *data class* ini memiliki beberapa properti:
  - ‘title’ dan ‘author’ menyimpan informasi teks berupa ‘String’
  - ‘coverImage’ dan ‘backgroundImage’ menyimpan referensi resource gambar bertipe ‘Int’
  - ‘url’ menyimpan alamat link Mangadex berupa ‘String’
  - ‘description’ dan ‘genres’ menyimpan kumpulan referensi resource dengan tipe ‘List<Int>‘
  - ‘id’ berfungsi sebagai referensi unik bertipe ‘Int’

Data class ini tetap sama dengan module 3 karena tidak ada perubahan struktur data apa pun.

MainActivity.kt (COMPOSE)

Pada baris [1] terdapat deklarasi package ‘com.example.modul4compose’, sama seperti file sebelumnya, yang menandakan lokasi file ini dalam struktur project.

Pada baris [3] sampai [11] terdapat kumpulan *directive import* yang digunakan untuk keperluan navigasi, tema, dan komponen Jetpack Compose.

Pada baris [12] sampai [45] terdapat class ‘MainActivity’ yang merupakan titik masuk utama aplikasi.

Pada baris [15] sampai [44] terdapat fungsi ‘onCreate()’ yang bertugas menyiapkan tampilan awal aplikasi. Di dalamnya terdapat beberapa hal:

- ‘enableEdgeToEdge()’ mengatur tampilan layar penuh
- ‘Modul3ComposeTheme’ membungkus seluruh tampilan dengan tema gelap
- ‘ComicViewModel’ disiapkan untuk menghubungkan View dengan data, sesuai peran ViewModel dalam arsitektur MVVM
- ‘isLandscape’ mendeteksi orientasi layar saat ini yang akan diturunkan kepada ‘screen’ atau ‘view’ yang menggunakannya.

- ‘NavHost’ mengatur navigasi antar screen, yaitu ‘MainScreen’ sebagai tampilan utama, ‘DetailsScreen’ untuk detail muncul saat diklik, dan ‘SettingsScreen’ yang hanya berisi opsi untuk mengganti bahasa EN ke ID atau sebaliknya.
- Perbedaan dari modul 3 adalah, penggunaan *factory* dengan mengubah

  ‘val viewModel: ComicViewModel = viewModel()’

  menjadi:

  ‘val viewModel: ComicViewModel = viewModel(

  factory = ComicViewModelFactory(

  appName = “MyComicList”,

  repository = ComicRepository()

)

)’

  Awalnya pada modul 3 instansiasi VM menggunakan ‘viewModel()’ melalui ‘ViewModelStoreOwner’, tetapi karena konstruktor ‘ComicViewModel()’ tidak lagi kosong (untuk string konstruktor & repository), maka perlu di instansiasi lewat ‘ViewModelFactory()’.

View.kt (COMPOSE)

Pada baris [1] terdapat deklarasi package ‘com.example.modul4compose’, sama seperti file sebelumnya, yang menandakan lokasi file ini dalam struktur project.

Pada baris [3] sampai [47] terdapat kumpulan *directive import* yang digunakan untuk keperluan navigasi, tema, dan komponen Jetpack Compose.

Pada baris [48] sampai [129] terdapat fungsi composable ‘RecommendationCarousel()’ yang menampilkan daftar komik dalam bentuk *carousel horizontal* menggunakan ‘LazyRow’ dengan ‘rememberSnapFlingBehavior’ agar tiap item tersangkut saat digeser. Latar belakang tiap item diambil dari ‘backgroundImage’ nya komik dari items yang diberi, lalu dilapisi gradasi hitam dari atas ke bawah menggunakan ‘Brush.verticalGradient’. Ukuran teks dan *layout* disesuaikan terlebih dahulu berdasarkan ‘isLandscape’, lalu diteruskan ke ‘ComicCard()’ yang bertugas menampilkan seluruh konten dan tata letak informasi komik.

Pada baris [130] sampai [243] terdapat fungsi composable ‘ComicList()’ yang menampilkan komik sebagai kartu individual secara vertikal. Sama seperti ‘RecommendationCarousel()’, ukuran teks dan *layout* disesuaikan terlebih dahulu berdasarkan ‘isLandscape’ sebelum diteruskan ke ‘ComicCard()’. Latar belakang kartu juga diambil dari ‘backgroundImage’ dan dilapisi gradasi hitam dari atas ke bawah menggunakan ‘Brush.verticalGradient’, dengan ‘ComicCard()’ sebagai konten utamanya. Selain itu, fungsi ini menambahkan bagian bawah berupa dua tombol, yaitu tombol yang membuka tautan komik di Mangadex melalui ‘Intent’, dan tombol yang menavigasi ke halaman detail melalui ‘navController’ sambil menyimpan komik yang dipilih melalui ‘viewModel.selectComic()’.

Pada baris [245] sampai [329] terdapat fungsi composable ‘ComicCard()’ yang merupakan komponen reusable yang digunakan oleh ‘RecommendationCarousel()’, ‘ComicList()’, dan fungsi ‘DetailsScreen()’ untuk menampilkan informasi komik. Tampilannya dibagi secara horizontal menggunakan ‘Row’, dengan gambar cover di sisi kiri dan informasi teks di sisi kanan yang mencakup judul, deskripsi, penulis, dan genre. Deskripsi dan genre masing-masing digabung dari ‘List<Int>‘ resource string menjadi satu teks menggunakan ‘joinToString()’. Gambar cover dan title dapat diklik untuk menavigasi ke halaman detail melalui ‘navController’ sambil menyimpan komik yang dipilih ke ‘viewModel’ melalui ‘selectComic’.

Bagian *view* tetap sama dengan module 3.

ViewModel.kt (COMPOSE)

Pada baris [1] terdapat deklarasi package ‘com.example.modul4compose’, sama seperti file sebelumnya, yang menandakan lokasi file ini dalam struktur project.

Pada baris [3] sampai [6] terdapat *directive import* untuk keperluan ‘ViewModel’ dan ‘MutableStateFlow’ dari Jetpack dan Kotlin Coroutines.

Pada baris [8] sampai [33] terdapat class ‘ComicViewModel’ yang merupakan lapisan ViewModel dalam arsitektur MVVM. Class ini menyimpan 3 *state* menggunakan ‘MutableStateFlow’, yaitu ‘_carouselComics’ dan ‘_listComics’ untuk menyimpan daftar komik, serta ‘_selectedComic’ untuk menyimpan komik yang sedang dipilih. Ketiganya diekspos secara publik sebagai ‘StateFlow’ read-only menggunakan ‘asStateFlow()’ agar View tidak bisa mengubah *state* secara langsung.

Pada baris [21] sampai [24] terdapat fungsi ‘selectComic()’ yang bertugas menyimpan komik yang dipilih pengguna ke dalam ‘_selectedComic’, yang kemudian dapat diakses oleh halaman detail.

Pada baris [26] sampai [32] terdapat blok ‘init’ yang berjalan saat ‘ComicViewModel’ pertama kali dibuat. Di dalamnya, ‘_carouselComics’ diisi dengan 7 data komik yang berasal dari ‘ComicRepository’ untuk ditampilkan di *carousel*, dan ‘_listComics’ diisi dengan 7 data komik lainnya yang berasal dari ‘ComicRepository’ untuk ditampilkan sebagai *list*. Tiap data komik dibuat menggunakan ‘data class Comic’ yang telah didefinisikan di ‘Model.kt’, dengan masing-masing properti seperti judul, resource gambar cover dan background, tautan, deskripsi, genre, dan penulis diisi secara langsung. Dalam balok ini terdapat juga logging menggunakan Timber.d(..) yang berguna untuk mencatat.

Perbedaan utama di class ini adalah memindahkan blok data yang di ‘init {}’ kepada ‘ComicRepository’ sehingga hanya perlu memanggil ‘getListComics’ dan ‘getCarouselComics’ dari reponya, selain itu sekarang ‘ComicViewModel()’ menerima konstruktur *string parameter* dan ‘ComicRepository()’.

Screen.kt (COMPOSE)

Pada baris [1] terdapat deklarasi package ‘com.example.modul4compose’, sama seperti file sebelumnya, yang menandakan lokasi file ini dalam struktur project.

Pada baris [3] sampai [43] terdapat kumpulan *directive import* untuk keperluan komponen UI Jetpack Compose, navigasi, dan lokalisasi.

Pada baris [45] sampai [96] terdapat fungsi composable ‘MainScreen()’ yang merupakan halaman utama aplikasi. Data komik diambil dari ‘viewModel’ menggunakan ‘collectAsStateWithLifecycle()’ untuk ‘carouselComics’ dan ‘listComics’. Halaman ini menggunakan ‘Scaffold’ dengan ‘TopAppBar’ yang menampilkan judul aplikasi dan tombol navigasi ke halaman settings. Konten utamanya menggunakan ‘LazyColumn’ yang berisi ‘RecommendationCarousel()’ di bagian atas, diikuti label, lalu daftar komik yang masing-masing ditampilkan menggunakan ‘ComicList()’.

Pada baris [98] sampai [160] terdapat fungsi composable ‘DetailsScreen()’ yang menampilkan detail komik yang dipilih. Komik yang dipilih diambil dari ‘viewModel’ melalui ‘selectedComic’. Ukuran teks disesuaikan berdasarkan ‘isLandscape’ sebelum

diteruskan ke ‘ComicCard()’. Sama seperti ‘RecommendationCarousel()’ dan ‘ComicList()’, latar belakang diambil dari ‘backgroundImage’ dan dilapisi gradasi hitam dari atas ke bawah, dengan ‘ComicCard()’ sebagai konten utamanya. Di bagian bawah terdapat tombol untuk kembali ke halaman sebelumnya menggunakan ‘navController.popBackStack()’.

Pada baris [162] sampai [209] terdapat fungsi composable ‘SettingsScreen()’ yang menampilkan halaman pengaturan bahasa. Bahasa aktif saat ini dideteksi melalui ‘AppCompatDelegate.getApplicationLocales()’, lalu bahasa yang akan diganti ditentukan secara otomatis, yaitu jika bahasa aktif adalah ‘en’ maka akan diganti ke ‘id’, dan sebaliknya. Halaman ini memiliki dua tombol, yaitu tombol untuk mengganti bahasa menggunakan ‘AppCompatDelegate.setApplicationLocales()’, dan tombol untuk kembali ke halaman sebelumnya menggunakan ‘navController.popBackStack()’.

Sebenarnya disini tidak ada perubahan, tetapi sebaiknya Screen.kt dipisah menjadi individual screen yang mungkin diorganisir secara *feature-based*, atau *task-based*, tetapi karena file sistem saya cukup simple maka saya biarkan.

ComicViewModelFactory.kt (COMPOSE)

Pada baris [1] terdapat deklarasi package ‘com.example.modul4compose’, sama seperti file sebelumnya, yang menandakan lokasi file ini dalam struktur project.

Pada baris [3] sampai [4] terdapat *directive import* untuk ‘ViewModel’ dan ‘ViewModelProvider’ dari Jetpack yang dibutuhkan untuk membuat *factory* class.

Pada baris [6] sampai [17] terdapat class ‘ComicViewModelFactory’ yang mengimplementasikan ‘ViewModelProvider.Factory’. Class ini menerima dua parameter konstruktor pada baris [7] dan [8], yaitu ‘appName’ bertipe ‘String’ dan ‘repository’ bertipe ‘ComicRepository’, yang nantinya akan diteruskan ke ‘ComicViewModel’ saat dibuat.

Pada baris [10] terdapat anotasi ‘@Suppress(“UNCHECKED_CAST”)’ yang digunakan untuk menekan peringatan *compiler* terkait operasi *cast* yang tidak bisa diverifikasi secara penuh pada saat kompilasi, secara spesifik adalah cast dari ‘return ComicViewModel(appName, repository) as T’ di bagian “as T”.

Pada baris [11] sampai [16] terdapat override fungsi ‘create()’ yang merupakan inti dari *factory* ini. Fungsi ini pada baris [12] mengecek apakah ‘modelClass’ yang diminta sesuai dengan ‘ComicViewModel’. Jika ya, maka pada baris [13] *instance* baru

‘ComicViewModel(appName, repository)’ dibuat dan dikembalikan dengan *cast* ke tipe generik ‘T’. Jika tidak sesuai, maka pada baris [15] dilempar ‘IllegalArgumentException’ dengan pesan ‘Unknown ViewModel’ sebagai penanda bahwa ViewModel yang diminta tidak dikenali oleh factory ini.

ComicRepository.kt (COMPOSE)

Pada baris [1] terdapat deklarasi package ‘com.example.modul4compose’, sama seperti file sebelumnya, yang menandakan lokasi file ini dalam struktur project.

Pada baris [4] sampai [94] terdapat *function* ‘getCarouselComics()’ yang merupakan *getter* yang berfungsi untuk me-return 7 data komik untuk diisi ke dalam carousel. Ini sebelumnya berada dalam ‘ViewModel()’ tetapi sekarang cukup memanggil function ini di init untuk mendapatkan data komiknya.

Pada baris [95] sampai [161] terdapat *function* ‘getListComics()’ yang merupakan *getter* yang berfungsi untuk me-return 7 data komik yang berbeda dari carousel untuk diisi ke dalam list. Ini sebelumnya berada dalam ‘ViewModel()’ tetapi sekarang cukup memanggil function ini di init untuk mendapatkan data komiknya.

MainActivity.kt (XML)

Pada baris [1] terdapat deklarasi package ‘com.example.modul4xml’, yang menandakan lokasi file ini dalam struktur project.

Pada baris [3] sampai [8] terdapat *directive import* untuk keperluan Activity, View Binding, dan WindowInsets.

Pada baris [10] sampai [26] terdapat class ‘MainActivity’ yang merupakan titik masuk utama aplikasi versi XML. Berbeda dengan versi Compose yang menginisialisasi seluruh navigasi dan ViewModel di dalam ‘onCreate()’, di sini ‘MainActivity’ hanya bertugas sebagai container, karena navigasi antar halaman dikelola oleh Fragment melalui ‘navHostFragment’. View Binding diinisialisasi melalui ‘ActivityMainBinding.inflate()’, lalu ‘setContentView()’ dipanggil dengan ‘binding.root’. Setelah itu, ‘ViewCompat.setOnApplyWindowInsetsListener()’ dipasangkan pada ‘navHostFragment’ untuk menyesuaikan padding berdasarkan system bars agar tampilan tidak tertutup status bar atau navigation bar.

Model.kt (XML)

Pada baris [1] terdapat deklarasi package ‘com.example.modul4compose’, yang menandakan lokasi file ini dalam package project.

Pada baris [3] sampai [12] terdapat ‘data classComic’, yang berfungsi sebagai Model dalam arsitektur MVVM, yaitu representasi data dari sebuah komik. Sesuai peran di MVVM, class ini hanya berisi struktur data.

- *data class* ini memiliki beberapa properti:
  - ‘title’ dan ‘author’ menyimpan informasi teks berupa ‘String’
  - ‘coverImage’ dan ‘backgroundImage’ menyimpan referensi resource gambar bertipe ‘Int’
  - ‘url’ menyimpan alamat link Mangadex berupa ‘String’
  - ‘description’ dan ‘genres’ menyimpan kumpulan referensi resource dengan tipe ‘List<Int>‘
  - ‘id’ berfungsi sebagai referensi unik bertipe ‘Int’

MainFragment.kt (XML)

Pada baris [1] terdapat deklarasi package ‘com.example.modul4xml.fragment’, sama seperti file sebelumnya, yang menandakan lokasi file ini dalam struktur project.

Pada baris [3] sampai [17] terdapat kumpulan *directive import* untuk keperluan Fragment, View Binding, navigasi, RecyclerView, dan adapter.

Pada baris [19] sampai [79] terdapat class ‘MainFragment’ dengan struktur binding dan pengambilan ‘viewModel’ yang mirip dengan ‘DetailsFragment’, menggunakan pola ‘binding’ dan ‘activityViewModels()’.

Pada baris [24] sampai [31] terdapat fungsi ‘onCreateView()’ yang menginisialisasi View Binding melalui ‘FragmentMainBinding.inflate()’ dan mengembalikan ‘binding.root’ sebagai tampilan fragment.

Pada baris [33] sampai [73] terdapat fungsi ‘onViewCreated()’ yang mengatur seluruh logika tampilan. Orientasi layar dideteksi melalui ‘resources.configuration.orientation’ dan tombol settings pada toolbar diatur untuk menavigasi ke halaman settings melalui ‘navController’. Carousel diimplementasikan

menggunakan ‘RecyclerView’ horizontal dengan ‘CarouselAdapter’, lalu ‘PagerSnapHelper’ dipasangkan agar tiap item tersangkut saat digeser, mirip dengan ‘rememberSnapFlingBehavior’ pada versi Compose. Jika orientasi landscape, tinggi carousel disesuaikan secara langsung melalui ‘LayoutParams’. Daftar komik diimplementasikan menggunakan ‘RecyclerView’ vertikal dengan ‘ComicListAdapter’, dengan ‘isNestedScrollingEnabled’ dinonaktifkan agar scrolling tidak bentrok dengan scroll utama. Kedua adapter kemudian diisi datanya melalui ‘viewModel.carouselComics.observe()’ dan ‘viewModel.listComics.observe()’ menggunakan ‘submitList()’.

Pada baris [75] sampai [78] terdapat fungsi ‘onDestroyView()’ yang menghapus referensi binding, sama seperti ‘DetailsFragment’.

Sama dengan ‘DetailsFragment’, perbedaan utama antara module 3 adalah karena soal 1.c, yaitu penggantian LiveData ke StateFlow, sehingga .observe diganti menjadi .collect. Contohnya sebagai berikut:

Lama:

```
‘viewModel.carouselComics.observe(viewLifecycleOwner) { comics ->
carouselAdapter.submitList(comics)’
```

Baru:

```
‘viewLifecycleOwner.lifecycleScope.launch {
viewLifecycleOwner.repeatOnLifecycle(Lifecycle.State.STARTED) {
viewModel.carouselComics.collect { comics ->

carouselAdapter.submitList(comics)
                }
        }
}’
```

Selain itu, karena sekarang instansiasi dilakukan menggunakan factory maka diganti juga ‘private val viewModel: ComicViewModel by activityViewModels()’ menjadi

```
‘private val viewModel: ComicViewModel by activityViewModels {
ComicViewModelFactory(
appName = “Modul 3 XML”,
```

```
repository = ComicRepository()
        )
}'
```

SettingsFragment.kt (XML)

Pada baris [1] terdapat deklarasi package 'com.example.modul4xml.fragment', sama seperti file sebelumnya, yang menandakan lokasi file ini dalam struktur project.

Pada baris [2] sampai [11] terdapat kumpulan *directive import* untuk keperluan Fragment, View Binding, navigasi, dan lokalisasi.

Pada baris [13] sampai [53] terdapat class 'SettingsFragment' dengan struktur binding yang serupa dengan fragment sebelumnya, menggunakan pola '_binding', namun tanpa 'activityViewModels()' karena halaman ini tidak memerlukan data dari 'viewModel'.

Pada baris [17] sampai [24] terdapat fungsi 'onCreateView()' yang menginisialisasi View Binding melalui 'FragmentSettingsBinding.inflate()' dan mengembalikan 'binding.root' sebagai tampilan fragment.

Pada baris [26] sampai [47] terdapat fungsi 'onViewCreated()' yang mengatur logika pergantian bahasa dan navigasi. Berbeda dengan versi Compose yang langsung membandingkan bahasa aktif dan menentukan bahasa baru dalam satu langkah, di sini bahasa aktif diambil terlebih dahulu melalui 'AppCompatDelegate.getApplicationLocales()', dengan fallback ke 'Locale.getDefault()' jika locale belum diatur. Selain itu, pengecekan bahasa Indonesia dilakukan untuk dua kode locale yaitu 'id' dan 'in' untuk memastikan kompatibilitas dengan kode bahasa Indonesia yang lama. Bahasa kemudian diganti menggunakan 'AppCompatDelegate.setApplicationLocales()'. Tombol kembali diatur untuk memanggil 'navController.popBackStack()'.

Pada baris [49] sampai [52] terdapat fungsi 'onDestroyView()' yang menghapus referensi binding, sama seperti fragment sebelumnya.

DetailsFragment.kt (XML)

Pada baris [1] terdapat deklarasi package 'com.example.modul4compose', yang menandakan lokasi file ini dalam package project.

Pada baris [3] sampai [18] terdapat kumpulan *directive import* untuk keperluan Fragment, View Binding, navigasi, locale etdan ViewModel.

Pada baris [20] sampai [110] terdapat class 'DetailsFragment' yang merupakan implementasi halaman detail dalam versi XML. Berbeda dengan versi Compose yang menggunakan 'selectedComic' dari 'StateFlow', di sini binding dikelola menggunakan 'FragmentDetailsBinding' dengan pola '_binding' yang dinullkan saat view dihancurkan, dan 'viewModel' diambil menggunakan 'activityViewModels()'.

Pada baris [30] sampai [37] terdapat fungsi 'onCreateView()' yang menginisialisasi View Binding melalui 'FragmentDetailsBinding.inflate()' dan mengembalikan 'binding.root' sebagai tampilan fragment.

Pada baris [39] sampai [98] terdapat fungsi 'onViewCreated()' yang mengatur seluruh logika tampilan. Orientasi layar dideteksi melalui 'resources.configuration.orientation', lalu digunakan untuk menyesuaikan ukuran teks, batas baris, dan bobot *layout* antara area kartu dan tombol secara langsung melalui 'LayoutParams'. Data komik diamati melalui 'viewModel.selectedComic.observe()', di mana setiap properti komik seperti gambar, judul, deskripsi, penulis, dan genre diisi langsung ke masing-masing view pada binding. Deskripsi dan genre digabung menggunakan 'joinToString()' serupa dengan versi Compose. Di bagian akhir, tombol kembali diatur untuk memanggil 'navController.popBackStack()'.

Pada baris [106] sampai [109] terdapat fungsi 'onDestroyView()' yang menghapus referensi binding dengan mengembalikan '_binding' ke null untuk menghindari memory leak.

Perbedaan utama antara module 3 adalah karena soal 1.c, yaitu penggantian LiveData ke StateFlow, sehingga .observe diganti menjadi .collect. Contohnya sebagai berikut:

Lama:

'viewModel.selectedComic.observe(viewLifecycleOwner) { comic ->

comic ?: return@observe

val ctx = requireActivity()'

Baru

'viewLifecycleOwner.lifecycleScope.launch {

viewLifecycleOwner.repeatOnLifecycle(Lifecycle.State.STARTED) {
viewModel.selectedComic.collect { comic ->
comic ?: return@collect
val ctx = requireActivity()'

Selain itu, karena sekarang instansiasi dilakukan menggunakan factory maka diganti juga 'private val viewModel: ComicViewModel by activityViewModels()' menjadi

'private val viewModel: ComicViewModel by activityViewModels {
ComicViewModelFactory(
appName = "Modul 3 XML",
repository = ComicRepository()
)
}'

ComicViewModel.kt (XML)

Pada baris [1] terdapat deklarasi package 'com.example.modul4xml.viewmodel', yang menandakan lokasi file ini dalam struktur project.

Pada baris [3] sampai [10] terdapat *directive import* untuk keperluan 'ViewModel' dan 'StateFlow dari Jetpack.

Pada baris [12] sampai [32] terdapat class 'ComicViewModel' yang strukturnya serupa dengan versi Compose, yaitu menyimpan 3 state untuk 'carouselComics', 'listComics', dan 'selectedComic', serta mengeksposnya sebagai read-only. Kali ini, versi XML menggunakan 'MutableStateFlow' dan 'StateFlow', sama dengan versi compose yang menggunakan 'MutableStateFlow' dan 'StateFlow', dan versi XML menggunakan 'collect()' untuk memantau perubahan data, sedikit berbeda dengan 'collectAsState()' karena versi XMLnya perlu di-wrap dengan 'repeatOnLifeCycle' agar lifecycle-safe, sedangkan collectAsState sudah melakukan itu dibalik layar.

Pada baris [24] sampai [26] terdapat fungsi 'selectComic()' yang bertugas menyimpan komik yang dipilih ke '_selectedComic', sama seperti versi Compose.

Pada baris [28] sampai [31] terdapat blok 'init' yang berjalan saat 'ComicViewModel' pertama kali dibuat. Di dalamnya, '_carouselComics' diisi dengan 7

data komik yang berasal dari ‘ComicRepository’ untuk ditampilkan di carousel, dan ‘_listComics’ diisi dengan 7 data komik lainnya yang berasal dari ‘ComicRepository’ untuk ditampilkan sebagai list. Tiap data komik dibuat menggunakan ‘data class Comic’ yang telah didefinisikan di ‘Model.kt’, dengan masing-masing properti seperti judul, resource gambar cover dan background, tautan, deskripsi, genre, dan penulis diisi secara langsung. Dalam balok ini terdapat juga logging menggunakan Timber.d(..) yang berguna untuk mencatat.

Perbedaan utama di class ini adalah memindahkan blok data yang di ‘init {}’ kepada ‘ComicRepository’ sehingga hanya perlu memanggil ‘getListComics’ dan ‘getCarouselComics’ dari reponya, selain itu sekarang ‘ComicViewModel()’ menerima konstruktur string parameter dan ‘ComicRepository()’.

CarouselAdapter.kt (XML)

Pada baris [1] terdapat deklarasi package ‘com.example.modul4xml.viewmodel’, yang menandakan lokasi file ini dalam struktur project.

Pada baris [3] sampai [12] terdapat *directive import* untuk keperluan RecyclerView, View Binding, navigasi, dan DiffUtil.

Pada baris [14] sampai [71] terdapat class ‘CarouselAdapter’ yang merupakan adapter untuk carousel pada versi XML, sebagai pengganti fungsi composable ‘RecommendationCarousel()’ di versi Compose. Class ini mewarisi ‘ListAdapter’ dengan ‘DiffCallback’ sebagai pembanding antar item, dan menerima ‘isLandscape’, ‘viewModel’, dan ‘navController’ sebagai parameter.

Pada baris [18] sampai [20] terdapat inner class ‘ViewHolder’ yang membungkus ‘ItemCarouselBinding’ sebagai representasi tampilan tiap item.

Pada baris [22] sampai [27] terdapat fungsi ‘onCreateViewHolder()’ yang menginisialisasi ‘ItemCarouselBinding’ untuk tiap item baru yang dibuat.

Pada baris [29] sampai [54] terdapat fungsi ‘onBindViewHolder()’ yang mengisi tampilan tiap item dengan data komik. Gambar latar dan cover diisi melalui ‘setImageResource()’, sedangkan teks judul, deskripsi, penulis, dan genre diisi langsung ke masing-masing view dengan ukuran teks dan batas baris yang disesuaikan berdasarkan ‘isLandscape’. Deskripsi dan genre digabung menggunakan ‘joinToString()’ serupa dengan versi Compose. Baik ‘coverImage’ maupun ‘infoColumn’ diatur agar dapat diklik untuk

menyimpan komik yang dipilih melalui ‘viewModel.selectComic()’ dan menavigasi ke halaman detail.

Pada baris [67] sampai [70] terdapat inner class ‘DiffCallback’ yang membandingkan item berdasarkan ‘id’ untuk ‘areItemsTheSame()’ dan keseluruhan objek untuk ‘areContentsTheSame()’, sehingga ‘ListAdapter’ hanya memperbarui item yang benar-benar berubah.

ComicListAdapter.kt (XML)

Pada baris [1] terdapat deklarasi package ‘com.example.modul4xml.adapter’, sama seperti file sebelumnya, yang menandakan lokasi file ini dalam struktur project.

Pada baris [3] sampai [14] terdapat *directive import* untuk keperluan RecyclerView, View Binding, navigasi, DiffUtil, dan Intent.

Pada baris [16] sampai [99] terdapat class ‘ComicListAdapter’ yang strukturnya serupa dengan ‘CarouselAdapter’, mewarisi ‘ListAdapter’ dengan ‘DiffCallback’ dan menerima parameter yang sama. Class ini merupakan pengganti fungsi composable ‘ComicList()’ di versi Compose.

Pada baris [22] terdapat inner class ‘ViewHolder’ yang membungkus ‘ItemComicListBinding’ sebagai representasi tampilan tiap item.

Pada baris [24] sampai [37] terdapat fungsi ‘onCreateViewHolder()’ yang menginisialisasi ‘ItemComicListBinding’ untuk tiap item. Berbeda dengan ‘CarouselAdapter’, fungsi ini juga langsung menyesuaikan tinggi kartu berdasarkan ‘isLandscape’ melalui ‘getDimensionPixelSize()’ dan mengubah ‘LayoutParams’ pada ‘binding.root’.

Pada baris [39] sampai [92] terdapat fungsi ‘onBindViewHolder()’ yang mengisi tampilan tiap item dengan cara yang serupa dengan ‘CarouselAdapter’, yaitu mengisi gambar, teks, dan ukuran font berdasarkan ‘isLandscape’. Selain itu, fungsi ini juga menambahkan pengecekan ‘currentDestination’ sebelum navigasi untuk memastikan navigasi hanya dilakukan dari ‘mainFragment’. Terdapat pula dua tombol tindakan, yaitu ‘btnMangadex’ yang membuka tautan komik melalui ‘Intent’, dan ‘btnDetail’ yang menavigasi ke halaman detail melalui ‘navController’.

Pada baris [94] sampai [97] terdapat inner class ‘DiffCallback’ yang identik dengan milik ‘CarouselAdapter’, membandingkan item berdasarkan ‘id’ dan keseluruhan objek.

ComicViewModelFactory.kt (XML)

Pada baris [1] terdapat deklarasi package ‘com.example. modul4xml’, sama seperti file sebelumnya, yang menandakan lokasi file ini dalam struktur project.

Pada baris [3] sampai [6] terdapat *directive import* untuk ‘ViewModel’ dan ‘ViewModelProvider’ dari Jetpack yang dibutuhkan untuk membuat *factory* class.

Pada baris [8] sampai [19] terdapat class ‘ComicViewModelFactory’ yang mengimplementasikan ‘ViewModelProvider.Factory’. Class ini menerima dua parameter konstruktor pada baris [9] dan [10], yaitu ‘appName’ bertipe ‘String’ dan ‘repository’ bertipe ‘ComicRepository’, yang nantinya akan diteruskan ke ‘ComicViewModel’ saat dibuat.

Pada baris [12] terdapat anotasi ‘@Suppress(“UNCHECKED_CAST”)’ yang digunakan untuk menekan peringatan *compiler* terkait operasi *cast* yang tidak bisa diverifikasi secara penuh pada saat kompilasi, secara spesifik adalah cast dari ‘return ComicViewModel(appName, repository) as T’ di bagian “as T”.

Pada baris [13] sampai [16] terdapat override fungsi ‘create()’ yang merupakan inti dari *factory* ini. Fungsi ini pada baris [14] mengecek apakah ‘modelClass’ yang diminta sesuai dengan ‘ComicViewModel’. Jika ya, maka pada baris [15] *instance* baru ‘ComicViewModel(appName, repository)’ dibuat dan dikembalikan dengan *cast* ke tipe generik ‘T’. Jika tidak sesuai, maka pada baris [17] dilempar ‘IllegalArgumentException’ dengan pesan ‘Unknown ViewModel’ sebagai penanda bahwa ViewModel yang diminta tidak dikenali oleh factory ini.

Class ini sama dengan versi composenya, namun di sini perlu mengimpor ‘ComicRepository()’ dan ‘ComicViewModel()’ sebelum membuat ‘ComicViewModelFactory()’

ComicRepository.kt (COMPOSE)

Pada baris [1] terdapat deklarasi package ‘com.example.modul4xml’, sama seperti file sebelumnya, yang menandakan lokasi file ini dalam struktur project.

Pada baris [4] sampai [83] terdapat *function* ‘getCarouselComics()’ yang merupakan *getter* yang berfungsi untuk me-return 7 data komik untuk diisi ke dalam carousel. Ini sebelumnya berada dalam ‘ViewModel()’ tetapi sekarang cukup memanggil function ini di init untuk mendapatkan data komiknya.

Pada baris [84] sampai [144] terdapat *function* ‘getListComics()’ yang merupakan *getter* yang berfungsi untuk me-return 7 data komik yang berbeda dari carousel untuk diisi ke dalam list. Ini sebelumnya berada dalam ‘ViewModel()’ tetapi sekarang cukup memanggil function ini di init untuk mendapatkan data komiknya.

Sebenarnya ini sama dengan versi composenya, tetapi saya malas menyamakannya (sudah terlanjur beda di module 3). Perbedaannya hanya pada perpindahan baris sehingga tidak konsekuensial.

Untuk .xml sendiri, tidak ada perubahan sama sekali dari modul 3 karena bukan termasuk scope untuk *refactoring* kodenya.

activity_main.xml

File ini merupakan layout utama yang digunakan oleh ‘MainActivity’. Isinya hanya sebuah ‘FragmentContainerView’ yang memenuhi seluruh layar, berfungsi sebagai container navigasi utama aplikasi. ‘NavHostFragment’ ditentukan melalui atribut ‘android:name’, graf navigasi dimuat melalui ‘app:navGraph’ yang mengarah ke ‘nav_graph’, dan ‘app:defaultNavHost’ diatur ke ‘true’ agar fragment ini menangani tombol back secara otomatis.

fragment_main.xml

File ini merupakan layout untuk ‘MainFragment’. Layout utamanya menggunakan ‘CoordinatorLayout’ agar komponen di dalamnya dapat berkoordinasi, khususnya antara ‘AppBarLayout’ dan konten saat discroll.

Di bagian atas terdapat ‘AppBarLayout’ yang membungkus ‘MaterialToolbar’ dengan id ‘toolbar’, menampilkan judul aplikasi dari ‘app_title’ dan menu dari ‘main_menu’ yang berisi tombol navigasi ke halaman settings, serupa dengan ‘TopAppBar’ pada versi Compose.

Di bawahnya terdapat ‘NestedScrollView’ dengan ‘appbar_scrolling_view_behavior’ agar konten tidak tertutup toolbar saat discroll. Di dalamnya terdapat ‘LinearLayout’ vertikal yang berisi tiga elemen secara berurutan. Pertama adalah ‘RecyclerView’ dengan id ‘carouselRecyclerView’ dengan tinggi tetap 300dp yang digunakan sebagai carousel, di mana tingginya dapat disesuaikan secara dinamis oleh ‘MainFragment’ saat landscape. Kedua adalah ‘TextView’ dengan id ‘sectionTitle’ sebagai label pemisah antara carousel dan daftar komik. Ketiga adalah ‘RecyclerView’ dengan id ‘listRecyclerView’ dengan ‘nestedScrollingEnabled’ dinonaktifkan agar scrolling tidak bentrok dengan ‘NestedScrollView’, serupa dengan pengaturan yang dilakukan di ‘MainFragment’.

fragment_details.xml

File ini merupakan layout untuk ‘DetailsFragment’. Layout utamanya menggunakan ‘FrameLayout’ dengan ‘fitsSystemWindows’ agar konten tidak tertutup system bars.

Di dalamnya terdapat 3 lapisan yang nested. Lapisan pertama adalah ‘ImageView’ dengan id ‘backgroundImage’ yang memenuhi seluruh layar dengan ‘scaleType’ centerCrop sebagai gambar latar. Lapisan kedua adalah ‘View’ yang dilapisi drawable ‘gradient_card_overlay’ sebagai gradasi gelap, serupa dengan ‘Brush.verticalGradient’ pada versi Compose. Lapisan ketiga adalah ‘LinearLayout’ vertikal dengan ‘weightSum’ 100 yang membagi ruang layar menjadi dua bagian berdasarkan weight, di mana weight tiap bagian disesuaikan secara dinamis oleh ‘DetailsFragment’ berdasarkan ‘isLandscape’.

Bagian pertama adalah ‘LinearLayout’ dengan id ‘comicCardArea’ yang menampung konten komik secara horizontal dengan ‘weightSum’ 10. Di dalamnya terdapat ‘ShapeableImageView’ dengan id ‘coverImage’ weight 4 di sisi kiri, dan ‘LinearLayout’ dengan id ‘infoColumn’ weight 6 di sisi kanan yang menampung ‘titleText’, ‘descriptionText’, ‘authorText’, dan ‘genreText’ secara vertikal. Struktur pembagian ini identik dengan ‘Row’ pada ‘ComicCard()’ di versi Compose.

Bagian kedua adalah ‘MaterialButton’ dengan id ‘btnReturn’ yang menempati sisa ruang di bagian bawah sebagai tombol kembali.

fragment_settings.xml

File ini merupakan layout untuk ‘SettingsFragment’. Layout utamanya menggunakan ‘LinearLayout’ vertikal dengan ‘weightSum’ 5 untuk mendistribusikan ruang layar secara proporsional.

Di dalamnya terdapat beberapa elemen yang disisipkan oleh komponen ‘Space’ sebagai pemberi jarak antar elemen. Elemen pertama adalah ‘TextView’ yang menampilkan label ‘label_change_language’ sebagai judul halaman. Di bawahnya terdapat dua ‘MaterialButton’, yaitu ‘btnSwitchLanguage’ untuk mengganti bahasa dan ‘btnReturn’ untuk kembali ke halaman sebelumnya. Masing-masing tombol dan jarak antar elemen mendapat weight tersendiri, tanpa menggunakan padding atau margin vertikal secara eksplisit, berbeda dengan versi Compose yang menggunakan ‘Arrangement.SpaceEvenly’.

Item_carousel.xml

File ini merupakan layout untuk tiap item pada ‘carouselRecyclerView’ yang digunakan oleh ‘CarouselAdapter’. Strukturnya serupa dengan ‘fragment_details.xml’, yaitu menggunakan ‘FrameLayout’ sebagai container dengan 3 lapisan yang ditumpuk, yaitu ‘ImageView’ dengan id ‘backgroundImage’ sebagai gambar latar, ‘View’ dengan drawable ‘gradient_card_overlay’ sebagai lapisan gradasi gelap, dan ‘LinearLayout’ vertikal sebagai konten utama.

Di dalam ‘LinearLayout’ vertikal tersebut terdapat ‘TextView’ untuk label rekomendasi di bagian atas, diikuti ‘LinearLayout’ horizontal dengan ‘weightSum’ 10 yang membagi konten menjadi dua bagian. Di sisi kiri terdapat ‘ShapeableImageView’ dengan id ‘coverImage’ weight 4, dan di sisi kanan terdapat ‘LinearLayout’ dengan id ‘infoColumn’ weight 6 yang menampung ‘titleText’, ‘descriptionText’, ‘authorText’, dan ‘genreText’ secara vertikal. Struktur pembagian ini identik dengan ‘fragment_details.xml’, dengan perbedaan bahwa file ini tidak memiliki tombol di bagian bawah.

Item_comic_list.xml

File ini merupakan layout untuk tiap item pada ‘listRecyclerView’ yang digunakan oleh ‘ComicListAdapter’. Layout utamanya menggunakan ‘MaterialCardView’ dengan tinggi awal 200dp, namun tinggi ini dapat disesuaikan secara dinamis oleh ‘ComicListAdapter’ berdasarkan ‘isLandscape’.

Di dalamnya terdapat ‘FrameLayout’ dengan tiga lapisan yang ditumpuk, serupa dengan ‘item_carousel.xml’ dan ‘fragment_details.xml’, yaitu ‘ImageView’ dengan id ‘backgroundImage’, lapisan gradasi gelap melalui drawable ‘gradient_card_overlay’, dan ‘LinearLayout’ vertikal dengan ‘weightSum’ 14 sebagai konten utama.’

Berbeda dengan ‘item_carousel.xml’, ‘LinearLayout’ vertikal ini dibagi menjadi dua bagian berdasarkan bobot. Bagian pertama berbobot 10 menampung konten komik secara horizontal dengan ‘weightSum’ 10, dengan struktur ‘coverImage’, ‘infoColumn’, ‘titleText’, ‘descriptionText’, ‘authorText’, dan ‘genreText’ yang identik dengan ‘item_carousel.xml’. Bagian kedua berbobot 4 adalah ‘LinearLayout’ horizontal berlatar hitam yang menampung dua ‘MaterialButton’, yaitu ‘btnMangadex’ dengan warna oranye dan ikon logo MangaDex, dan ‘btnDetail’ dengan ikon menu, masing-masing berbobot sama sehingga membagi ruang secara merata.

SOAL 2

Jelaskan Application class dalam arsitektur aplikasi Android dan fungsinya

A. Jawaban:

Application class adalah *class* utama pada arsitektur aplikasi Android yang merepresentasikan keseluruhan aplikasi. *Class* ini dibuat sebelum *Activity*, *Fragment*, atau komponen UI lain dijalankan, sehingga cocok digunakan untuk melakukan inisialisasi global yang dibutuhkan oleh seluruh aplikasi.

Fungsi *Application class* adalah sebagai tempat konfigurasi awal aplikasi, seperti menyiapkan *library*, *dependency*, *logging*, *analytics*, *database*, dll. Karena *Application class* hanya dibuat satu kali selama proses aplikasi berjalan, *class* ini sering digunakan untuk setup yang bersifat umum dan dipakai oleh banyak bagian aplikasi, artinya hal yang bersifat general.

Dalam penggunaan umum, *Application class* tidak digunakan untuk menyimpan logika UI atau data sementaranua. Logika tampilan tetap diletakkan di *Activity*, *Fragment*, atau *ViewModel*. *Application class* sebaiknya hanya digunakan untuk kebutuhan inisialisasi aplikasi secara global.

Pada aplikasi ini, *Application class* digunakan untuk menginisialisasi *library* Timber. Timber perlu dipasang sekali saat aplikasi pertama kali berjalan dengan menggunakan 'Timber.plant(Timber.DebugTree())'. Dengan meletakkan kode tersebut di *Application class*, seluruh bagian aplikasi seperti *ViewModel*, *Activity*, *Fragment*, atau *Composable* dapat menggunakan 'Timber.d()' untuk melakukan logging tanpa perlu menginisialisasi Timber berulang kali.

Class ini bisa dikatakan mirip dengan *Singleton* karena hanya memiliki satu instance selama proses aplikasi berjalan, tetapi *Application Class* bukan *Singleton* manual, karena lifecycle dan pembuatannya tetap dikendalikan oleh sistem Android. Timber juga bisa diinstansiasi menggunakan *Singleton*, tetapi tidak disarankan karena hanya berperan sebagai wrapper, dan Timber sendiri sudah bersifat global melalui 'Timber.d(..)'. Contoh terakhir, Timber juga bisa diinstansiasi sebagai Singleton dan Application Class secara bersamaan, tetapi sebaiknya jangan dilakukan jika tidak disengajakan.

TAUTAN GITHUB

https://github.com/SebastianAbeSantoso/MODUL-MOBILE/tree/main/MODUL%204